# Bazele Inteligenţei Artificiale

Prof. Dr. Călin-Adrian POPA

## Cursul 6

4 Noiembrie 2025

# 6 Multilayer perceptrons
## 6.1 From linear models to deep neural networks

- in Chapter 1, we only talked about linear models
- while *neural networks* cover a much richer family of models, we can begin thinking of the linear model as a neural network by expressing it in the language of neural networks
- to begin, let us start by rewriting things in a “layer” notation
- diagrams are used in deep learning to visualize models
- in Figure 1, we depict the linear regression model as a neural network
- note that these diagrams highlight the connectivity pattern, such as how each input is connected to the output, but not the values taken by the weights or biases

# 6.1 From linear models to deep neural networks

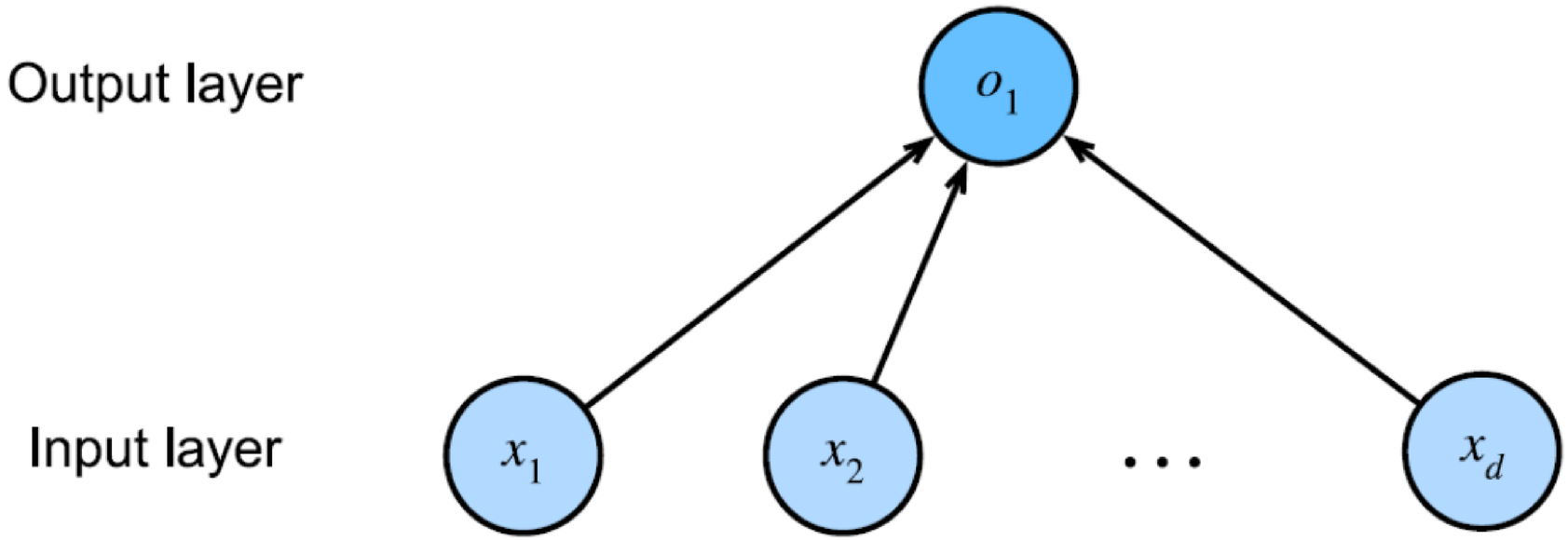



Figure 1: Linear regression is a single-layer neural network.

# 6.1 From linear models to deep neural networks

- for the neural network shown in Figure 1, the inputs are $x_1, \ldots, x_d$, so the *number of inputs* (or *feature dimensionality*) in the input layer is $d$
- the output of the network in Figure 1 is $o_1$, so the *number of outputs* in the output layer is 1
- note that the input values are all *given* and there is just a single *computed* neuron
- focusing on where computation takes place, conventionally, we do not consider the input layer when counting layers
- that is to say, the *number of layers* for the neural network in Figure 1 is 1
- we can think of linear regression models as neural networks consisting of just a single artificial neuron, or as *single-layer neural networks*

# 6.1 From linear models to deep neural networks

- since, for linear regression, every input is connected to every output (in this case, there is only one output), we can regard this transformation (the output layer in Figure 1) as a *fully-connected layer* or *dense layer*
- we will talk a lot more about networks composed of such layers in this and the following chapters
- since linear regression (invented in 1795) predates computational neuroscience, linear regression was not originally described as a neural network
- to see why linear models were a natural place to begin when the cyberneticists/neurophysiologists Warren McCulloch and Walter Pitts began to develop models of artificial neurons, consider the cartoonish picture of a biological neuron in Figure 2, consisting of *dendrites* (input terminals), the *nucleus* (CPU), the *axon* (output wire), and the *axon terminals* (output terminals), enabling connections to other neurons via *synapses*

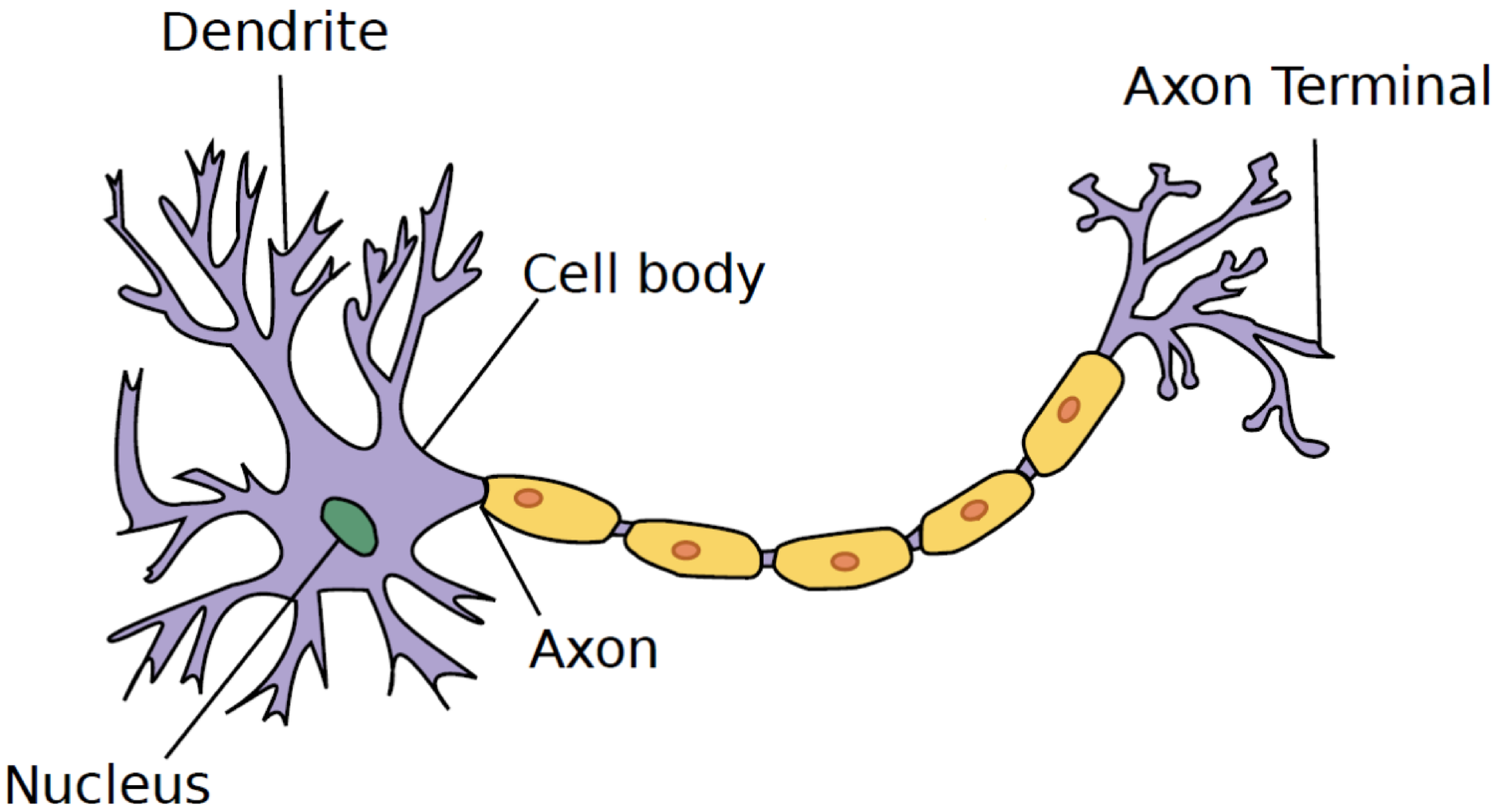



Figure 2: The real neuron.

# 6.1 From linear models to deep neural networks

- information $x_i$ arriving from other neurons (or environmental sensors, such as the retina) is received in the dendrites
- in particular, that information is weighted by *synaptic weights* $w_i$, determining the effect of the inputs (e.g., activation or inhibition via the product $x_i w_i$)
- the weighted inputs arriving from multiple sources are aggregated in the nucleus as a weighted sum $y = \sum_{i=1}^{d} x_i w_i + b$, and this information is then sent for further processing in the axon $y$, typically after some nonlinear processing via $\sigma(y)$
- from there, it either reaches its destination (e.g., a muscle) or is fed into another neuron via its dendrites

# 6.1 From linear models to deep neural networks

- certainly, the high-level idea that many such units could be stitched together with the right connectivity and right learning algorithm, to produce far more interesting and complex behavior than any one neuron alone could express, owes to our study of real biological neural systems
- at the same time, most research in deep learning today draws little direct inspiration from neuroscience
- although airplanes might have been *inspired* by birds, ornithology has not been the primary driver of aeronautics innovation for many years
- likewise, inspiration in deep learning these days comes in equal or greater measure from mathematics, statistics, and computer science

# 6.1 From linear models to deep neural networks

- *softmax regression* can also be framed as a neural network
- let us start off with a simple image classification problem; here, each input consists of a $2 \times 2$ grayscale image
- we can represent each pixel value with a single scalar, giving us four features $x_1, x_2, x_3, x_4$
- further, let us assume that each image belongs to one among the categories “cat”, “chicken”, and “dog”
- next, we have to choose how to represent the labels; we have two obvious choices
- perhaps the most natural impulse would be to choose $y \in \{1, 2, 3\}$, where the integers represent $\{\text{dog}, \text{cat}, \text{chicken}\}$, respectively; this is a great way of *storing* such information on a computer
- if the categories had some natural ordering among them, say if we were trying to predict $\{\text{baby}, \text{toddler}, \text{adolescent}, \text{young adult}, \text{adult}, \text{geriatric}\}$, then it might even make sense to cast this problem as regression, and keep the labels in this format

- but general classification problems do not come with natural orderings among the classes
- fortunately, statisticians long ago invented a simple way to represent categorical data: the *one-hot encoding*
- a one-hot encoding is a vector with as many components as we have categories
- the component corresponding to a particular instance's category is set to 1, and all other components are set to 0
- in our case, a label $y$ would be a three-dimensional vector, with $(1, 0, 0)$ corresponding to "cat", $(0, 1, 0)$ to "chicken", and $(0, 0, 1)$ to "dog":

$$y \in \{(1, 0, 0), (0, 1, 0), (0, 0, 1)\}.$$

# 6.1 From linear models to deep neural networks

- in order to estimate the conditional probabilities associated with all the possible classes, we need a model with multiple outputs, one per class
- to address classification with linear models, we will need as many affine functions as we have outputs
- each output will correspond to its own affine function
- in our case, since we have 4 features and 3 possible output categories, we will need 12 scalars to represent the weights ($w$ with subscripts), and 3 scalars to represent the biases ($b$ with subscripts)
- we compute these three *logits*, $o_1$, $o_2$, and $o_3$, for each input:

$$\begin{aligned} o_1 &= x_1 w_{11} + x_2 w_{12} + x_3 w_{13} + x_4 w_{14} + b_1, \\ o_2 &= x_1 w_{21} + x_2 w_{22} + x_3 w_{23} + x_4 w_{24} + b_2, \\ o_3 &= x_1 w_{31} + x_2 w_{32} + x_3 w_{33} + x_4 w_{34} + b_3. \end{aligned}$$

# 6.1 From linear models to deep neural networks

- we can depict this calculation with the neural network diagram shown in Figure 3
- just as in linear regression, softmax regression is also a *single-layer neural network*
- and, since the calculation of each output, $o_1$, $o_2$, and $o_3$, depends on all inputs, $x_1$, $x_2$, $x_3$, and $x_4$, the output layer of softmax regression can also be described as a fully-connected layer

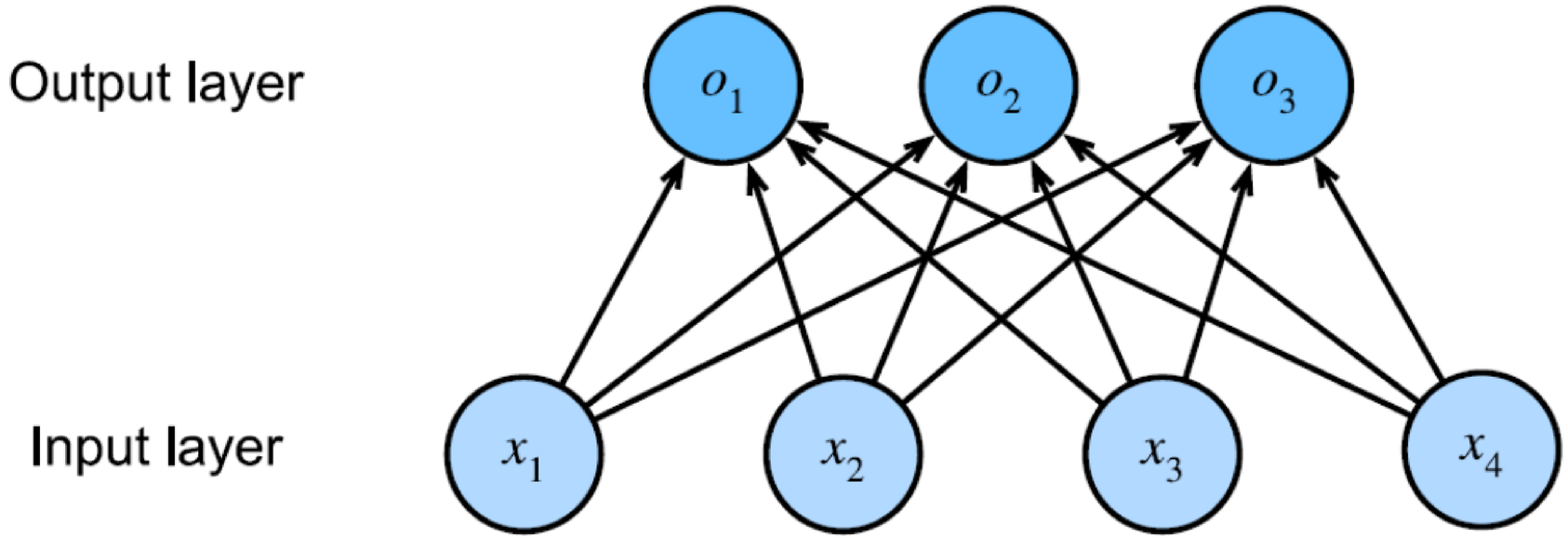



Figure 3: Softmax regression is a single-layer neural network.

# 6.1 From linear models to deep neural networks

- to express the model more compactly, we use the linear algebra notation
- in vector form, we arrive at $\boldsymbol{o} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$, a form better suited both for mathematics, and for writing code
- note that we have gathered all of our weights into a $3 \times 4$ matrix, and that, for features of a given data example $\boldsymbol{x}$, our outputs are given by a matrix-vector product of our weights by our input features, plus our biases $\boldsymbol{b}$
- as we will see in this and subsequent chapters, fully-connected layers are ubiquitous in deep learning
- however, as the name suggests, fully-connected layers are *fully* connected, with potentially many learnable parameters
- specifically, for any fully-connected layer with $d$ inputs and $q$ outputs, the parameterization cost is $\mathcal{O}(dq)$, which can be prohibitively high in practice

# 6.1 From linear models to deep neural networks

- as we discussed in Chapter 1, we want to interpret the outputs of our model as probabilities
- we do this by applying the *softmax function* to the logits vector $\boldsymbol{o}$:

$$\hat{\boldsymbol{y}} = \text{softmax}(\boldsymbol{o}) \qquad \text{where} \qquad \hat{y}_j = \frac{\exp(o_j)}{\sum_{j'=1}^{q} \exp(o_{j'})}.$$

- it is easy to see that $\sum_{j=1}^{q} \hat{y}_j = 1$, with $0 \leq \hat{y}_j \leq 1$, for all $1 \leq j \leq q$
- thus, $\hat{\boldsymbol{y}}$ is a proper *probability distribution*, whose element values can be interpreted accordingly
- note that the softmax operation does not change the ordering among the logits $\boldsymbol{o}$, which are simply the pre-softmax values that determine the probabilities assigned to each class
- therefore, during prediction, we can still pick out the most likely class by:

$$\underset{j}{\text{argmax}}\, \hat{y}_j = \underset{j}{\text{argmax}}\, o_j.$$

# 6.1 From linear models to deep neural networks

- although softmax is a nonlinear function, the outputs of softmax regression are still *determined* by an affine transformation of input features; thus, softmax regression is a linear model
- assume that we are given a design matrix $\boldsymbol{X}$ of $n$ examples with feature dimensionality (number of inputs) $d$
- assume that we have $q$ categories in the output
- then, the design matrix $\boldsymbol{X} \in \mathbb{R}^{n \times d}$, weights $\boldsymbol{W} \in \mathbb{R}^{d \times q}$, and the bias satisfies $\boldsymbol{b} \in \mathbb{R}^{1 \times q}$
- thus, we have:

$$\begin{aligned} \boldsymbol{O} &= \boldsymbol{X}\boldsymbol{W} + \boldsymbol{b}, \\ \hat{\boldsymbol{Y}} &= \text{softmax}(\boldsymbol{O}). \end{aligned}$$

- this accelerates the dominant operation into a matrix-matrix product $\boldsymbol{X}\boldsymbol{W}$ vs. the matrix-vector products we would be executing if we processed one example at a time
- since each row in $\boldsymbol{X}$ represents a data example, the softmax operation itself can be computed *row-wise*: for each row of $\boldsymbol{O}$, exponentiate all entries and then normalize them by the sum
- triggering *broadcasting* during the summation $\boldsymbol{X}\boldsymbol{W} + \boldsymbol{b}$, both the logits $\boldsymbol{O} \in \mathbb{R}^{n \times q}$ and output probabilities $\hat{\boldsymbol{Y}} \in \mathbb{R}^{n \times q}$ are $n \times q$ matrices

# 6.2 Multilayer perceptrons

- now, we will introduce our first truly *deep* network
- the simplest deep networks are called *multilayer perceptrons*, and they consist of multiple layers of neurons, each fully connected to those in the layer below (from which they receive input) and those above (which they, in turn, influence)
- we have described the affine transformation, which is a linear transformation added by a bias, in Chapter 1
- consider the model architecture corresponding to our softmax regression example, illustrated in Figure 3

# 6.2 Multilayer perceptrons

- this model mapped our inputs directly to our outputs via a single affine transformation, followed by a softmax operation
- if our labels truly were related to our input data by an affine transformation, then this approach would be sufficient
- but linearity in affine transformations is a *strong* assumption
- for example, linearity implies the weaker assumption of monotonicity: that any increase in our feature must either always cause an increase in our model's output (if the corresponding weight is positive), or always cause a decrease in our model's output (if the corresponding weight is negative); sometimes that makes sense

# 6.2 Multilayer perceptrons

- but what about classifying images of cats and dogs?
- should increasing the intensity of the pixel at location $(13, 17)$ always increase (or always decrease) the likelihood that the image depicts a dog?
- reliance on a linear model corresponds to the implicit assumption that the only requirement for differentiating cats vs. dogs is to assess the brightness of individual pixels
- this approach is doomed to fail in a world where inverting an image preserves the category

# 6.2 Multilayer perceptrons

- and yet, despite the apparent absurdity of linearity here, it is not obvious that we could address the problem with a simple preprocessing fix
- that is because the significance of any pixel depends in complex ways on its context (the values of the surrounding pixels)
- while there might exist a representation of our data that would take into account the relevant interactions among our features, on top of which a linear model would be suitable, we simply do not know how to calculate it by hand
- with deep neural networks, we use training data to jointly learn both a representation via *hidden layers* and a *linear predictor* that acts upon that representation

# 6.2 Multilayer perceptrons

- we can overcome these limitations of linear models and handle a more general class of functions by incorporating one or more hidden layers
- the easiest way to do this is to stack many fully-connected layers on top of each other
- each layer feeds into the layer above it, until we generate outputs
- we can think of the first $L - 1$ layers as our *representation* and the final layer as our *linear predictor*, which can be a linear regressor (for regression) or a logistic/softmax regressor (for classification)
- this architecture is commonly called a *multilayer perceptron*, often abbreviated as MLP
- below, we depict the diagram of an MLP (Figure 4)

# 6.2 Multilayer perceptrons

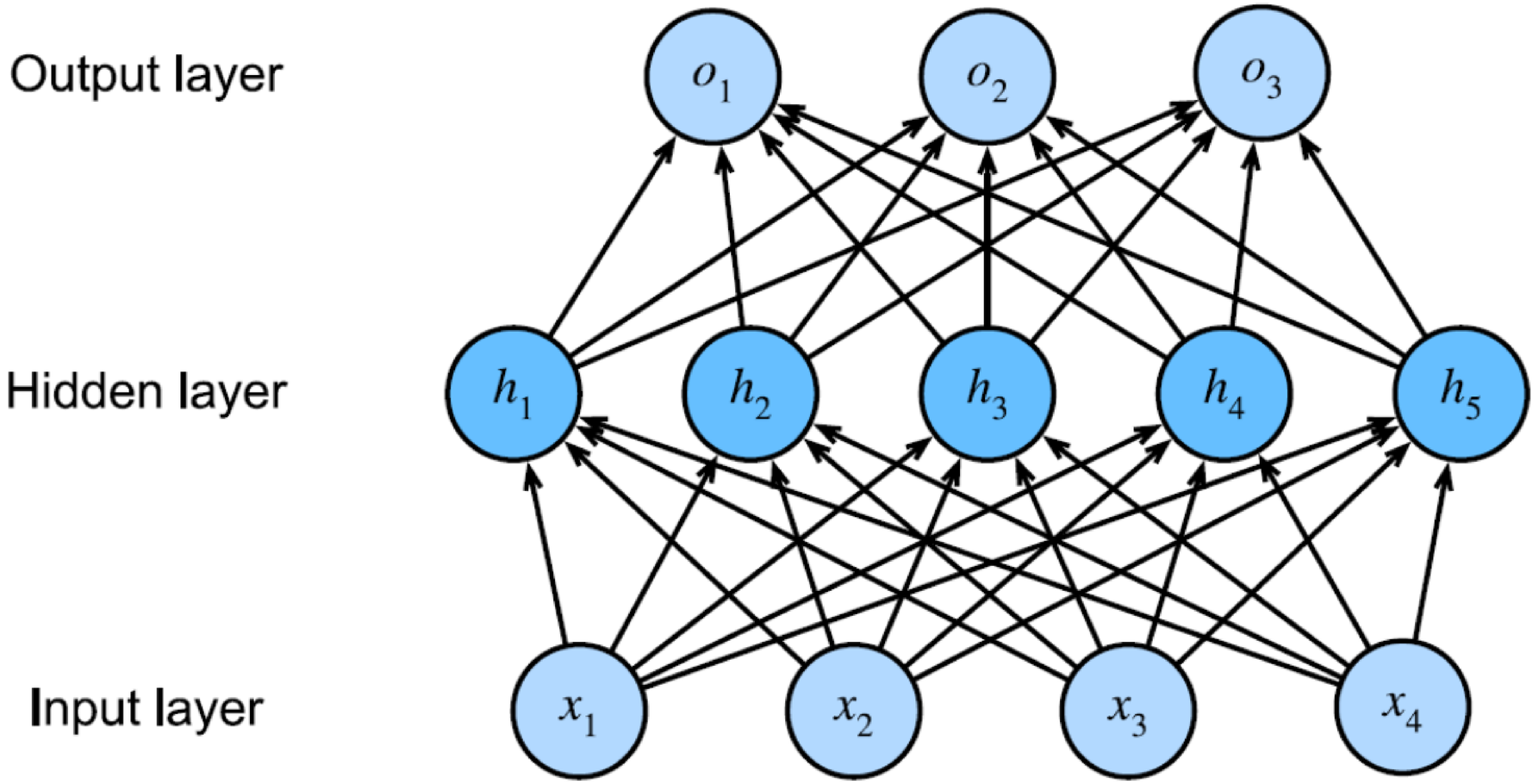



Figure 4: An MLP with a hidden layer of 5 hidden units.

# 6.2 Multilayer perceptrons

- this MLP has 4 inputs, 3 outputs, and its hidden layer contains 5 hidden units
- since the input layer does not involve any calculations, producing outputs with this network requires implementing the computations for both the hidden and output layers; thus, the number of layers in this MLP is 2
- note that these layers are both *fully connected*
- every input influences every neuron in the hidden layer, and each of these in turn influences every neuron in the output layer

# 6.2 Multilayer perceptrons

- as before, by the matrix $\boldsymbol{X} \in \mathbb{R}^{n \times d}$, we denote a design matrix of $n$ examples, where each example has $d$ inputs (features)
- for a one-hidden-layer MLP whose hidden layer has $h$ hidden units, denote by $\boldsymbol{H} \in \mathbb{R}^{n \times h}$ the outputs of the hidden layer, which are *hidden representations*
- in mathematics or code, $\boldsymbol{H}$ is also known as a *hidden-layer variable* or a *hidden variable*
- since the hidden and output layers are both fully connected, we have hidden-layer weights $\boldsymbol{W}^{(1)} \in \mathbb{R}^{d \times h}$ and biases $\boldsymbol{b}^{(1)} \in \mathbb{R}^{1 \times h}$ and output-layer weights $\boldsymbol{W}^{(2)} \in \mathbb{R}^{h \times q}$ and biases $\boldsymbol{b}^{(2)} \in \mathbb{R}^{1 \times q}$
- formally, we calculate the outputs $\boldsymbol{O} \in \mathbb{R}^{n \times q}$ of the one-hidden-layer MLP as follows:

$$\begin{aligned} \boldsymbol{H} &= \boldsymbol{X}\boldsymbol{W}^{(1)} + \boldsymbol{b}^{(1)}, \\ \boldsymbol{O} &= \boldsymbol{H}\boldsymbol{W}^{(2)} + \boldsymbol{b}^{(2)}. \end{aligned}$$

# 6.2 Multilayer perceptrons

- note that after adding the hidden layer, our model now requires us to track and update additional sets of parameters
- so what have we gained in exchange?
- we might be surprised to find out that – in the model defined above – *we gain nothing for our troubles*
- the reason is plain: the hidden units above are given by an affine function of the inputs, and the outputs (pre-softmax) are just an affine function of the hidden units
- an affine function of an affine function is itself an affine function
- moreover, our linear model was already capable of representing any affine function

# 6.2 Multilayer perceptrons

- we can view the equivalence formally, by proving that, for any values of the weights, we can just collapse out the hidden layer, yielding an equivalent single-layer model with parameters $\boldsymbol{W} = \boldsymbol{W}^{(1)}\boldsymbol{W}^{(2)}$ and $\boldsymbol{b} = \boldsymbol{b}^{(1)}\boldsymbol{W}^{(2)} + \boldsymbol{b}^{(2)}$:

$$\boldsymbol{O} = (\boldsymbol{X}\boldsymbol{W}^{(1)} + \boldsymbol{b}^{(1)})\boldsymbol{W}^{(2)} + \boldsymbol{b}^{(2)} = \boldsymbol{X}\boldsymbol{W}^{(1)}\boldsymbol{W}^{(2)} + \boldsymbol{b}^{(1)}\boldsymbol{W}^{(2)} + \boldsymbol{b}^{(2)} = \boldsymbol{X}\boldsymbol{W} + \boldsymbol{b}.$$

- in order to realize the potential of multilayer architectures, we need one more key ingredient: a *nonlinear activation function* $\sigma$ to be applied to each hidden unit, following the affine transformation
- the outputs of activation functions (e.g., $\sigma(\cdot)$) are called *activations*
- in general, with activation functions in place, it is no longer possible to collapse our MLP into a linear model:

$$\begin{aligned} \boldsymbol{H} &= \sigma(\boldsymbol{X}\boldsymbol{W}^{(1)} + \boldsymbol{b}^{(1)}), \\ \boldsymbol{O} &= \boldsymbol{H}\boldsymbol{W}^{(2)} + \boldsymbol{b}^{(2)}. \end{aligned}$$

# 6.2 Multilayer perceptrons

- since each row in $\boldsymbol{X}$ corresponds to an example, with some abuse of notation, we define the *nonlinearity* $\sigma$ to apply to its inputs in a row-wise fashion, i.e., one example at a time
- note that we used the notation for softmax in the same way to denote a row-wise operation before
- often, the activation functions that we apply to hidden layers are not merely row-wise, but element-wise
- that means that, after computing the linear portion of the layer, we can calculate each activation without looking at the values taken by the other hidden units; this is true for most activation functions
- to build more general MLPs, we can continue stacking such hidden layers, e.g., $\boldsymbol{H}^{(1)} = \sigma_1(\boldsymbol{X}\boldsymbol{W}^{(1)} + \boldsymbol{b}^{(1)})$ and $\boldsymbol{H}^{(2)} = \sigma_2(\boldsymbol{H}^{(1)}\boldsymbol{W}^{(2)} + \boldsymbol{b}^{(2)})$, one on top of the other, yielding ever more expressive models

# 6.2 Multilayer perceptrons

- MLPs can capture complex interactions among our inputs via their hidden neurons, which depend on the values of each of the inputs
- we can easily design hidden nodes to perform arbitrary computation, for instance, basic logic operations on a pair of inputs
- moreover, for certain choices of the activation function, it is widely known that MLPs are *universal approximators*
- even with a single-hidden-layer network, given enough nodes (possibly absurdly many), and the right set of weights, we can model any function, though actually learning that function is the hard part
- we might think of our neural network as being a bit like the C programming language
- the language, like any other modern language, is capable of expressing *any* computable program
- but actually coming up with a program that meets our specifications is the hard part

# 6.2 Multilayer perceptrons

- moreover, just because a single-hidden-layer network *can* learn any function, does not mean that we should try to solve all of our problems with single-hidden-layer networks
- in fact, we can approximate many functions much more compactly by using deeper (vs. wider) networks
- activation functions decide whether a neuron should be activated or not, by calculating the weighted sum and further adding bias to it
- they are differentiable operators to transform input signals to outputs, while most of them add a nonlinearity
- because activation functions are fundamental to deep learning, let us briefly survey some common activation functions

# 6.2 Multilayer perceptrons

- the most popular choice, due to both simplicity of implementation and its good performance on a variety of predictive tasks, is the *rectified linear unit* (ReLU)
- ReLU provides a very simple nonlinear transformation
- given an element $x$, the function is defined as the maximum of that element and 0:

$$\text{ReLU}(x) = \max(x, 0).$$

- informally, the ReLU function retains only positive elements and discards all negative elements by setting the corresponding activations to 0
- to gain some intuition, we plot the function in Figure 5; as we can see, the activation function is *piecewise linear*

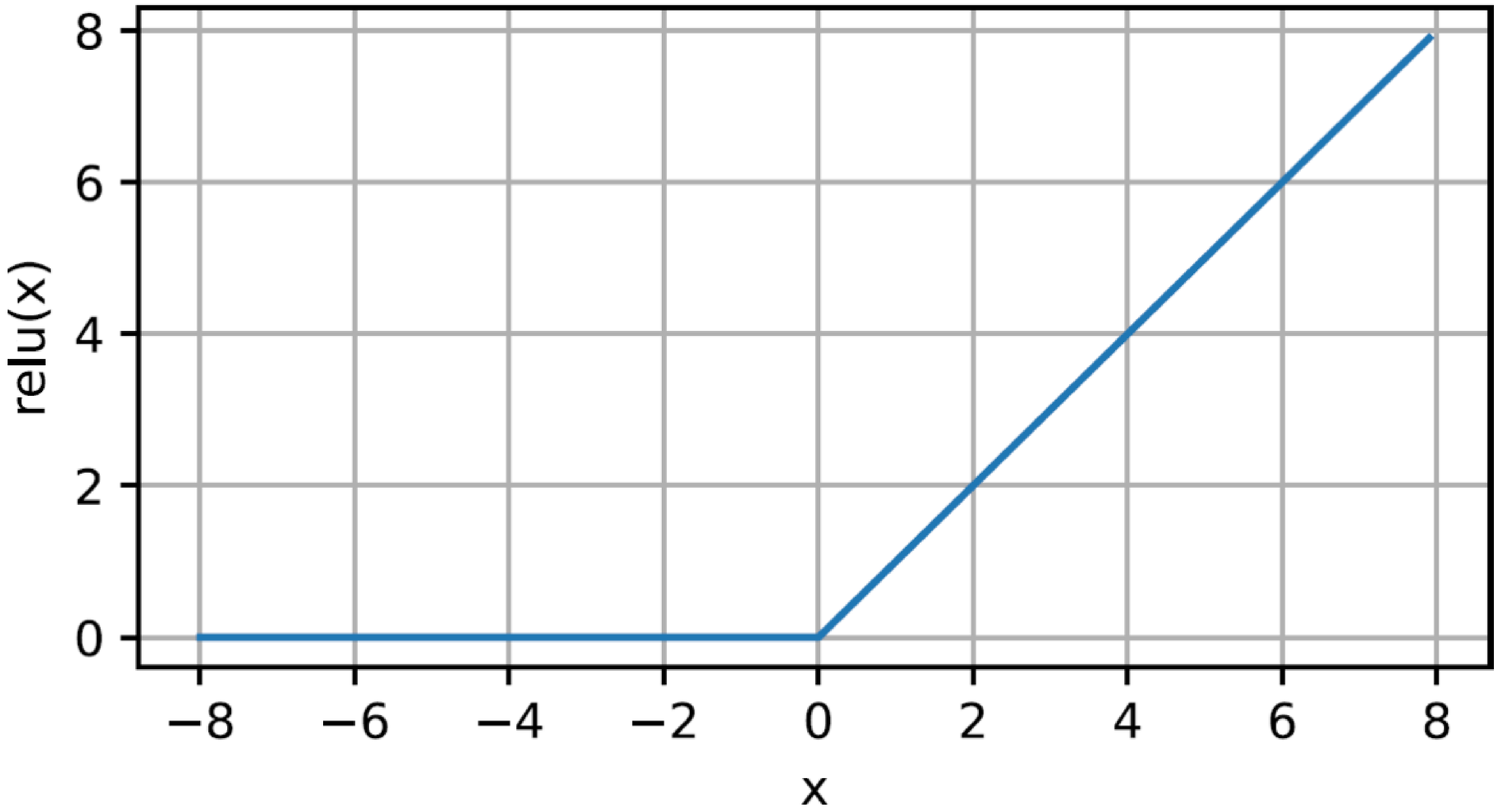



Figure 5: The ReLU function.

# 6.2 Multilayer perceptrons

- when the input is negative, the derivative of the ReLU function is 0, and when the input is positive, the derivative of the ReLU function is 1
- note that the ReLU function is not differentiable when the input takes value precisely equal to 0
- in these cases, we default to the left-hand-side derivative and say that the derivative is 0 when the input is 0
- we can get away with this, because the input may never actually be zero
- we plot the derivative of the ReLU function in Figure 6

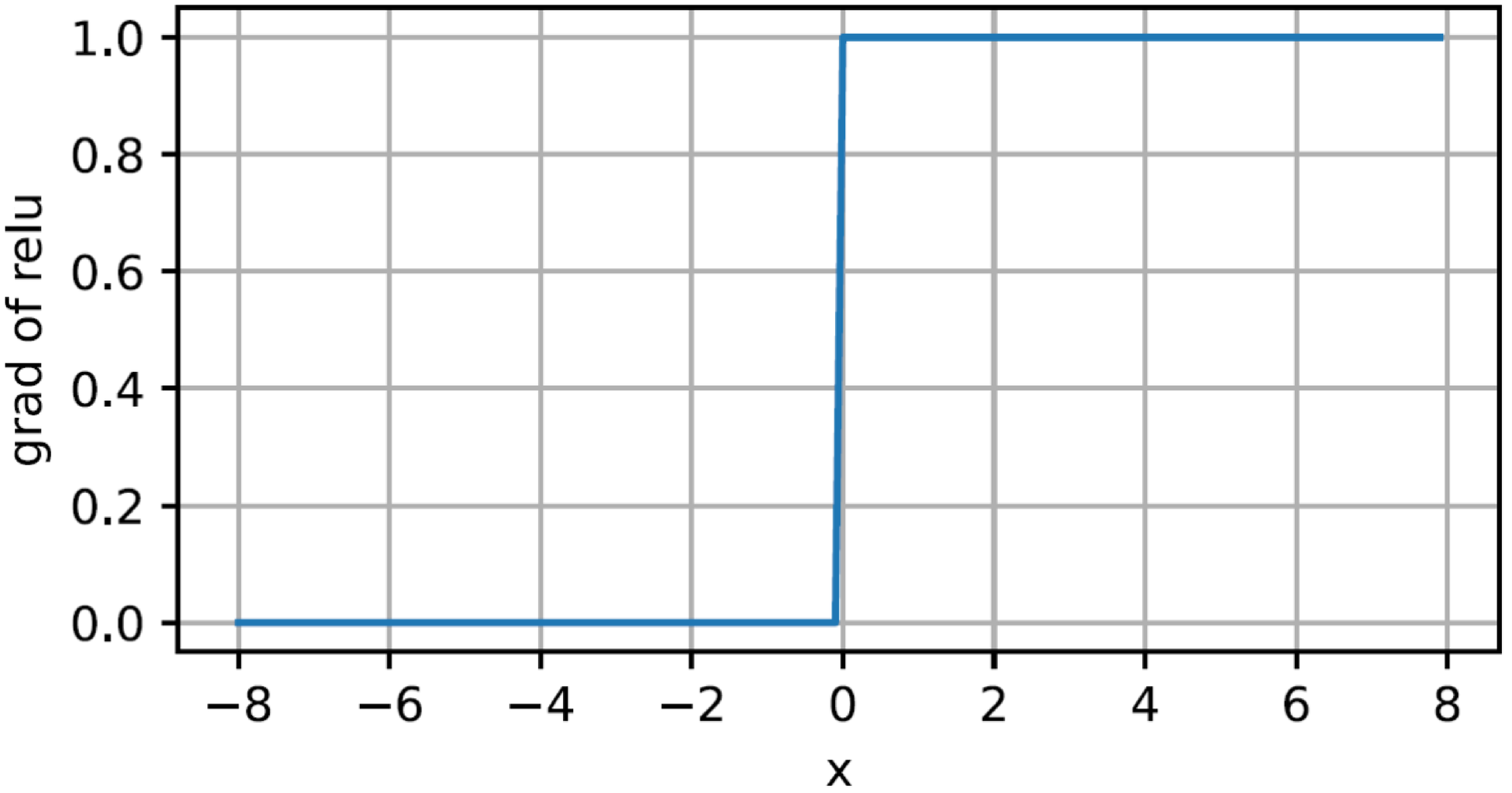



Figure 6: The derivative of the ReLU function.

- the reason for using ReLU is that its derivatives are particularly well behaved: either they vanish or they just let the argument through
- this makes optimization better behaved and it mitigates the well-documented problem of vanishing gradients that plagued previous versions of neural networks (more on this later)
- note that there are many variants of the ReLU function, including the *parameterized ReLU* (pReLU) function
- this variation adds a linear term to ReLU, so some information still gets through, even when the argument is negative:

$$\text{pReLU}(x) = \max(0, x) + \alpha \min(0, x).$$

# 6.2 Multilayer perceptrons

- the *sigmoid function* transforms its inputs, for which values lie in the domain $\mathbb{R}$, to outputs that lie in the interval $(0, 1)$
- for that reason, the sigmoid is often called a *squashing function*: it squashes any input in the range $(-\infty, \infty)$ to some value in the range $(0, 1)$:

$$\text{sigmoid}(x) = \frac{1}{1 + \exp(-x)}.$$

- in the earliest neural networks, scientists were interested in modeling biological neurons which either *fire* or *do not fire*
- thus, the pioneers of this field, going all the way back to McCulloch and Pitts, the inventors of the artificial neuron, focused on *thresholding units*
- a thresholding activation takes value 0 when its input is below some threshold, and value 1 when the input exceeds the threshold

# 6.2 Multilayer perceptrons

- when attention shifted to gradient based learning, the sigmoid function was a natural choice because it is a smooth, differentiable approximation to a thresholding unit
- sigmoids are still widely used as activation functions on the output units, when we want to interpret the outputs as probabilities for *binary classification* problems (we can think of the sigmoid as a special case of the softmax)
- however, the sigmoid has mostly been replaced by the simpler and more easily trainable ReLU for most use in hidden layers
- in the chapter on recurrent neural networks, we will describe architectures that leverage sigmoid units to control the flow of information across time
- we plot the sigmoid function in Figure 7; note that, when the input is close to 0, the sigmoid function approaches a linear transformation

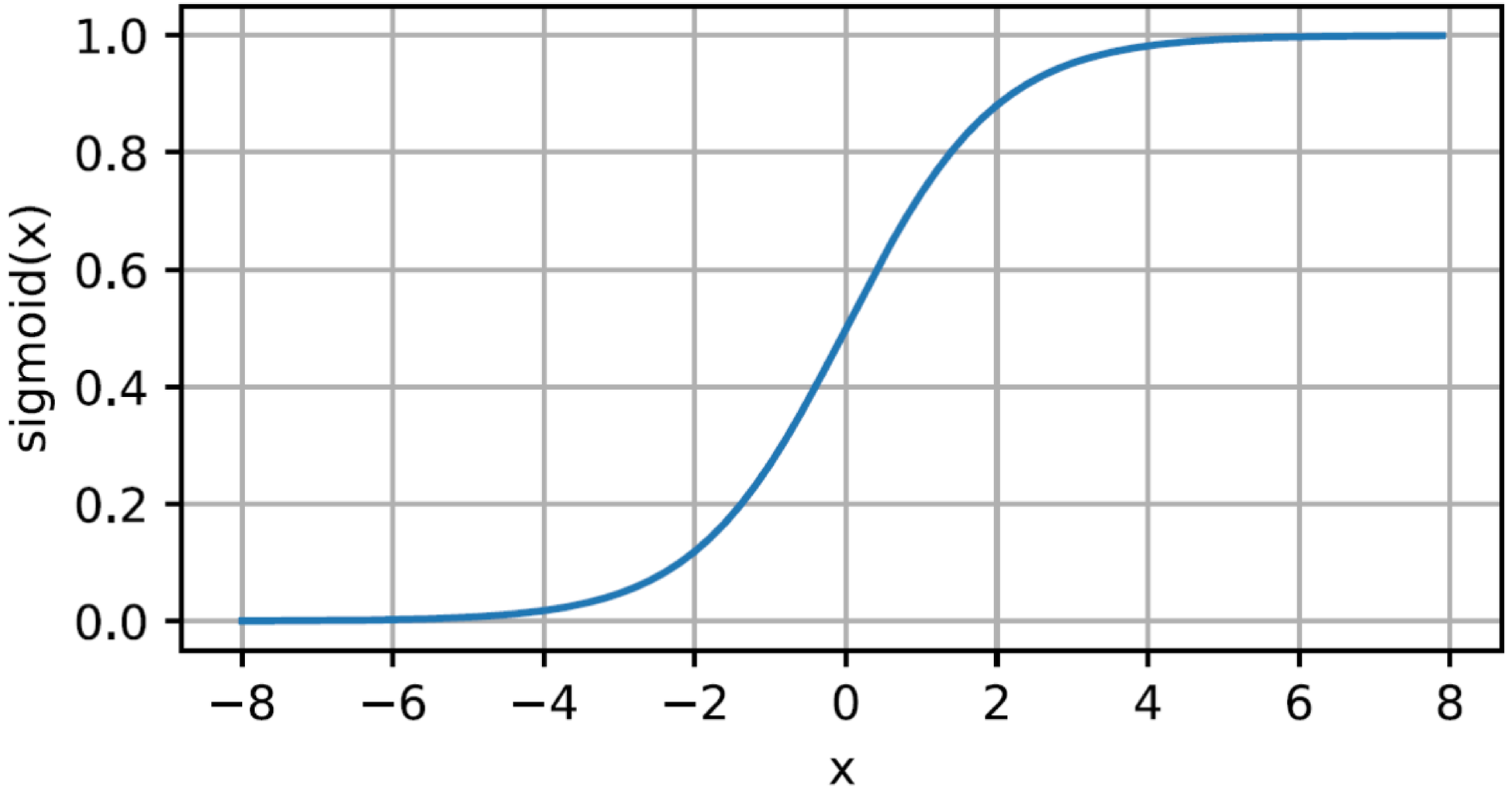



Figure 7: The sigmoid function.

# 6.2 Multilayer perceptrons

- the derivative of the sigmoid function is given by the following equation:

$$\frac{d}{dx}\text{sigmoid}(x) = \frac{\exp(-x)}{(1+\exp(-x))^2} = \text{sigmoid}(x)(1-\text{sigmoid}(x)).$$

- the derivative of the sigmoid function is plotted in Figure 8
- note that when the input is 0, the derivative of the sigmoid function reaches a maximum of 0.25
- as the input diverges from 0 in either direction, the derivative approaches 0

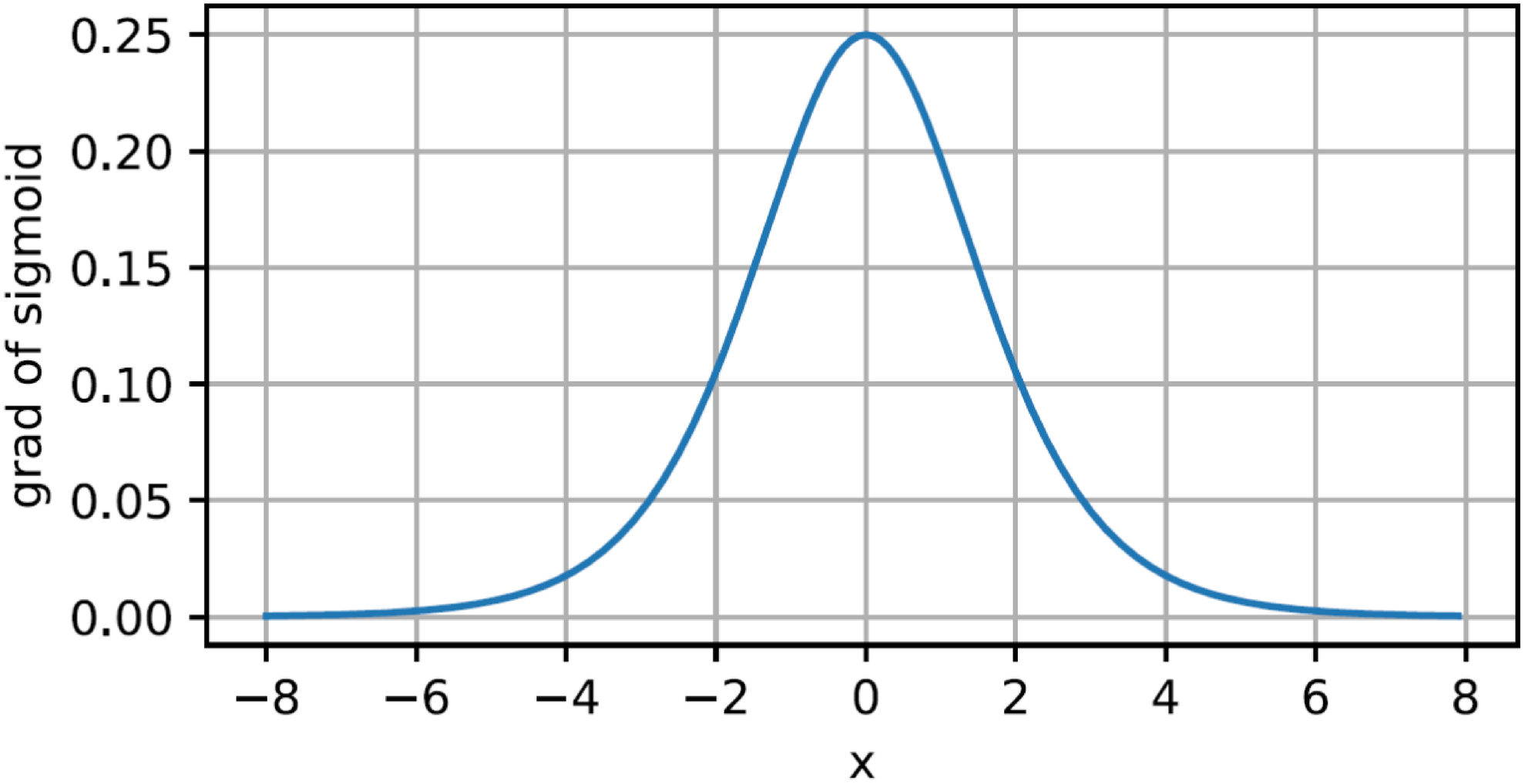



Figure 8: The derivative of the sigmoid function.

# 6.2 Multilayer perceptrons

- like the sigmoid function, the *tanh* (*hyperbolic tangent*) function also squashes its inputs, transforming them into elements in the interval between $-1$ and 1:

$$\tanh(x) = \frac{1 - \exp(-2x)}{1 + \exp(-2x)}.$$

- we plot the tanh function in Figure 9
- note that, as the input nears 0, the tanh function approaches a linear transformation
- although the shape of the function is similar to that of the sigmoid function, the tanh function exhibits point symmetry about the origin of the coordinate system

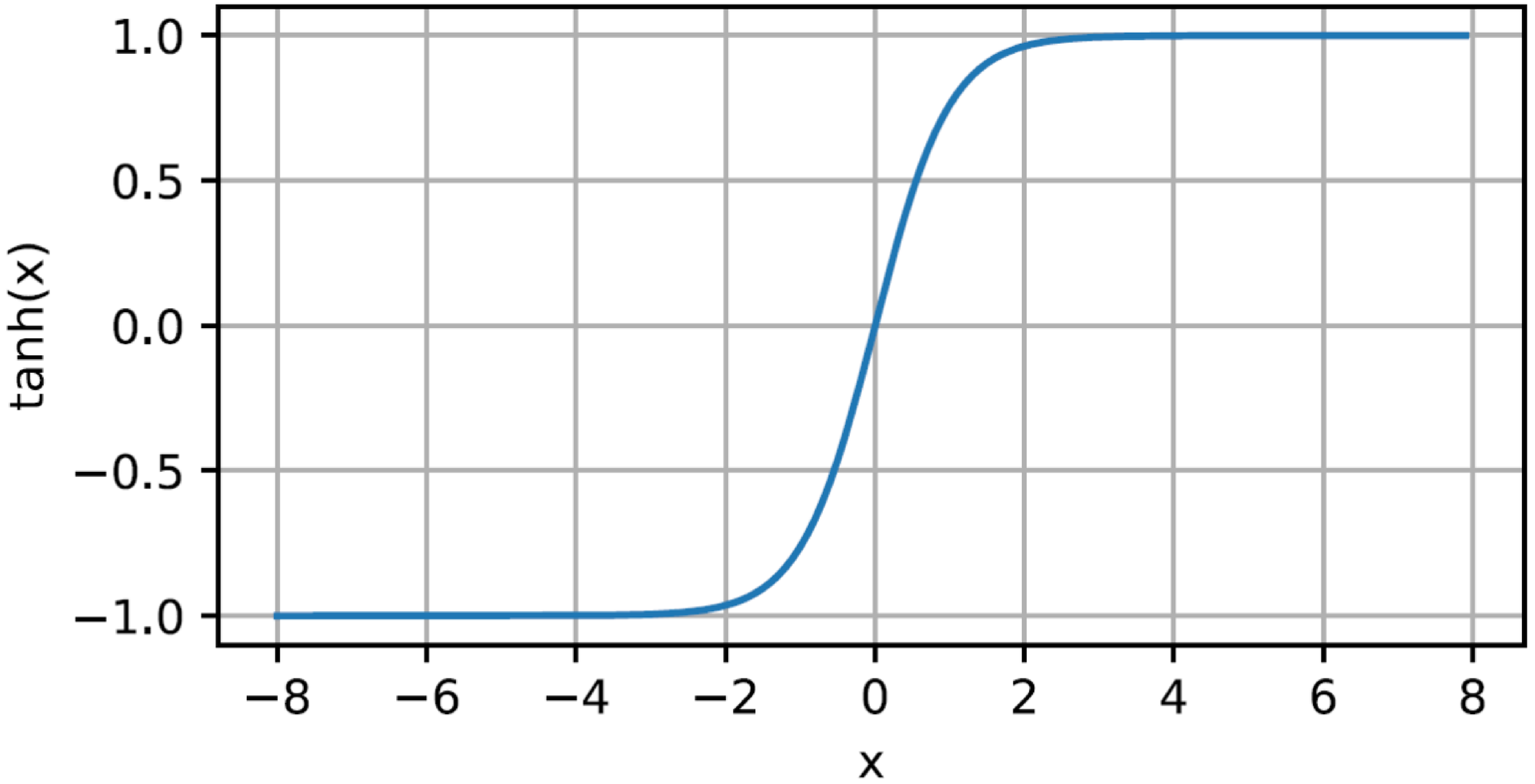



Figure 9: The tanh function.

- the derivative of the tanh function is:

$$\frac{d}{dx}\tanh(x) = 1 - \tanh^2(x).$$

- the derivative of tanh function is plotted in Figure 10
- as the input nears 0, the derivative of the tanh function approaches a maximum of 1
- and, as we saw with the sigmoid function, as the input moves away from 0 in either direction, the derivative of the tanh function approaches 0

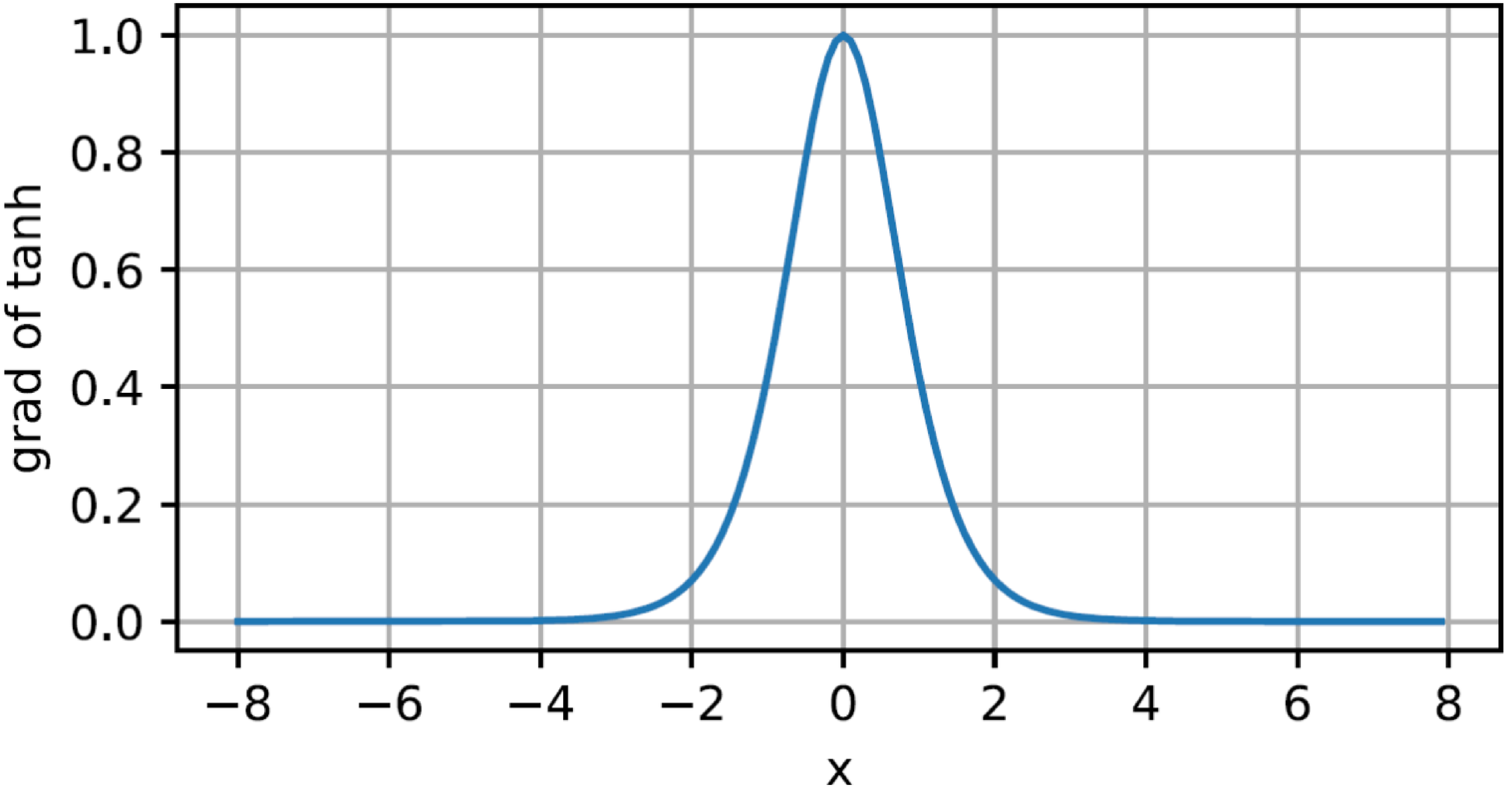



Figure 10: The derivative of the tanh function.

# 6.2 Multilayer perceptrons

- in summary, we now know how to incorporate nonlinearities to build expressive multilayer neural network architectures
- as a side note, our knowledge already puts us in command of a similar toolkit to a practitioner circa 1990
- in some ways, we have an advantage over anyone working in the 1990s, because we can leverage powerful open-source deep learning frameworks to build models rapidly, using only a few lines of code
- previously, training these networks required researchers to code up thousands of lines of C and Fortran

# 6.3 Model selection, underfitting, and overfitting

- in machine learning, the goal is to discover *patterns*
- but how can we be sure that we have truly discovered a *general* pattern and not simply memorized our data?
- our goal is to discover patterns that capture regularities in the underlying population from which our training set was drawn
- if we are successful in this endeavor, then we could successfully assess the performance even for samples that we have never encountered before
- this problem – how to discover patterns that *generalize* – is the fundamental problem of machine learning

# 6.3 Model selection, underfitting, and overfitting

- the danger is that, when we train models, we access just a small sample of data
- the largest public image datasets contain roughly one million images
- more often, we must learn from only thousands or tens of thousands of data examples
- when working with finite samples, we run the risk that we might discover apparent associations that turn out not to hold up when we collect more data
- the phenomenon of fitting our training data more closely than we fit the underlying distribution is *overfitting*, and the techniques used to combat overfitting are *regularization* techniques

# 6.3 Model selection, underfitting, and overfitting

- in order to discuss this phenomenon more formally, we need to differentiate between training error and generalization error
- the *training error* is the error of our model as calculated on the training dataset, while *generalization error* is the expectation of our model's error were we to apply it to an infinite stream of additional data examples drawn from the same underlying data distribution as our original sample
- problematically, we can never calculate the generalization error exactly
- that is because the stream of infinite data is an imaginary object
- in practice, we must *estimate* the generalization error by applying our model to an independent test set constituted of a random selection of data examples that were withheld from our training set

# 6.3 Model selection, underfitting, and overfitting

- in the standard supervised learning setting, we assume that both the training data and the test data are drawn *independently* from *identical* distributions
- this is commonly called the *i.i.d. assumption*, which means that the process that samples our data has no memory
- in other words, the second example drawn and the third drawn are no more correlated than the second and the two-millionth sample drawn
- there are common cases where this assumption fails; however, we will assume that the i.i.d. assumption holds

# 6.3 Model selection, underfitting, and overfitting

- when we train our models, we attempt to search for a function that fits the training data as well as possible
- if the function is so flexible that it can catch on to spurious patterns just as easily as to true associations, then it might perform *too well*, without producing a model that generalizes well to unseen data
- this is precisely what we want to avoid or at least control
- many of the techniques in deep learning are heuristics and tricks aimed at guarding against overfitting

# 6.3 Model selection, underfitting, and overfitting

- when we have simple models and abundant data, we expect the generalization error to resemble the training error
- when we work with more complex models and fewer examples, we expect the training error to go down, but the *generalization gap* to grow
- what precisely constitutes model complexity is a complex matter
- many factors govern whether a model will generalize well
- for example, a model with *more parameters* might be considered more complex
- a model whose parameters can take a *wider range of values* might be more complex
- often with neural networks, we think of a model that takes *more training iterations* as more complex, and one subject to *early stopping* (fewer training iterations) as less complex

# 6.3 Model selection, underfitting, and overfitting

- it can be difficult to compare the complexity among members of substantially different model classes (say, decision trees vs. neural networks)
- for now, a simple rule of thumb is quite useful: a model that can readily explain arbitrary facts is what can be viewed as complex, whereas one that has only a limited expressive power, but still manages to explain the data well, is probably closer to the truth
- this conforms to the principle of Occam's razor: given two explanations for something, the explanation most likely to be correct is the simplest one – the one that makes fewer assumptions

# 6.3 Model selection, underfitting, and overfitting

- next, we will focus on a few factors that tend to influence the generalizability of a model class:
  1. The number of tunable parameters. When the number of tunable parameters, sometimes called the *degrees of freedom*, is large, models tend to be more susceptible to overfitting.
  2. The values taken by the parameters. When weights can take a wider range of values, models can be more susceptible to overfitting.
  3. The number of training examples. It is trivially easy to overfit a dataset containing only one or two examples, even if our model is simple. But overfitting a dataset with millions of examples requires an extremely flexible model.

# 6.3 Model selection, underfitting, and overfitting

- in machine learning, we usually select our final model after evaluating several candidate models; this process is called *model selection*
- sometimes, the models subject to comparison are fundamentally different in nature (say, decision trees vs. linear models)
- at other times, we are comparing members of the same class of models that have been trained with *different hyperparameter settings*
- with MLPs, for example, we may wish to compare models with different numbers of hidden layers, different numbers of hidden units, and various choices of the activation functions applied to each hidden layer
- in order to determine the best among our candidate models, we will typically employ a validation dataset

# 6.3 Model selection, underfitting, and overfitting

- in principle, we should not touch our test set until after we have chosen all our hyperparameters
- were we to use the test data in the model selection process, there is a risk that we might *overfit the test data*, which is very problematic
- if we overfit our training data, there is always the evaluation on test data to keep us honest
- but if we overfit the test data, how would we ever know?
- thus, we should never rely on the test data for model selection
- and yet we cannot rely solely on the training data for model selection either, because we cannot estimate the generalization error on the very data that we use to train the model

# 6.3 Model selection, underfitting, and overfitting

- the common practice to address this problem is to split our data *three ways*, incorporating a *validation set*, in addition to the training and test sets
- the performance on this validation set will be used for model selection
- when training data is scarce, we might not even be able to afford to hold out enough data to constitute a proper validation set
- one popular solution to this problem is to use *K-fold cross-validation*
- here, the original training data is split into $K$ non-overlapping subsets
- then, model training and validation are executed $K$ times, each time training on $K - 1$ subsets and validating on a different subset (the one not used for training in that round)
- finally, the training and validation errors are estimated by averaging over the results from the $K$ experiments

# 6.3 Model selection, underfitting, and overfitting

- when we compare the training and validation errors, we should be aware of two common situations
- first, we should watch out for cases when our training error and validation error are both substantial, but there is a *little gap* between them
- if the model is unable to reduce the training error, that could mean that our model is too simple (i.e., insufficiently expressive) to capture the pattern that we are trying to model
- moreover, since the *generalization gap* between our training and validation errors is small, we have reason to believe that we could use a more complex model
- this phenomenon is known as *underfitting*

# 6.3 Model selection, underfitting, and overfitting

- on the other hand, as we discussed above, we should watch out for the cases when our training error is significantly lower than our validation error, indicating severe *overfitting*
- note that overfitting is not always a bad thing
- with deep learning especially, it is well known that the best predictive models often perform far better on training data than on holdout data
- ultimately, we usually care more about the validation error than about the gap between the training and validation errors

# 6.3 Model selection, underfitting, and overfitting

- whether we overfit or underfit can depend both on the complexity of our model and the size of the available training datasets, two topics that we discuss below
- to illustrate some classical intuition about overfitting and model complexity, we give an example using polynomials
- given training data consisting of a single feature $x$ and a corresponding real-valued label $y$, we try to find the polynomial of degree $d$ to estimate the labels $y$

# 6.3 Model selection, underfitting, and overfitting

- a higher-order polynomial function is more complex than a lower-order polynomial function, since the higher-order polynomial has more parameters and the model function's selection range is wider
- fixing the training dataset, higher-order polynomial functions should always achieve lower (at worst, equal) training error relative to lower degree polynomials
- in fact, whenever the data examples each have a distinct value of $x$, a polynomial function with degree equal to the number of data examples can fit the training set perfectly
- we visualize the relationship between polynomial degree and underfitting vs. overfitting in Figure 11

# 6.3 Model selection, underfitting, and overfitting

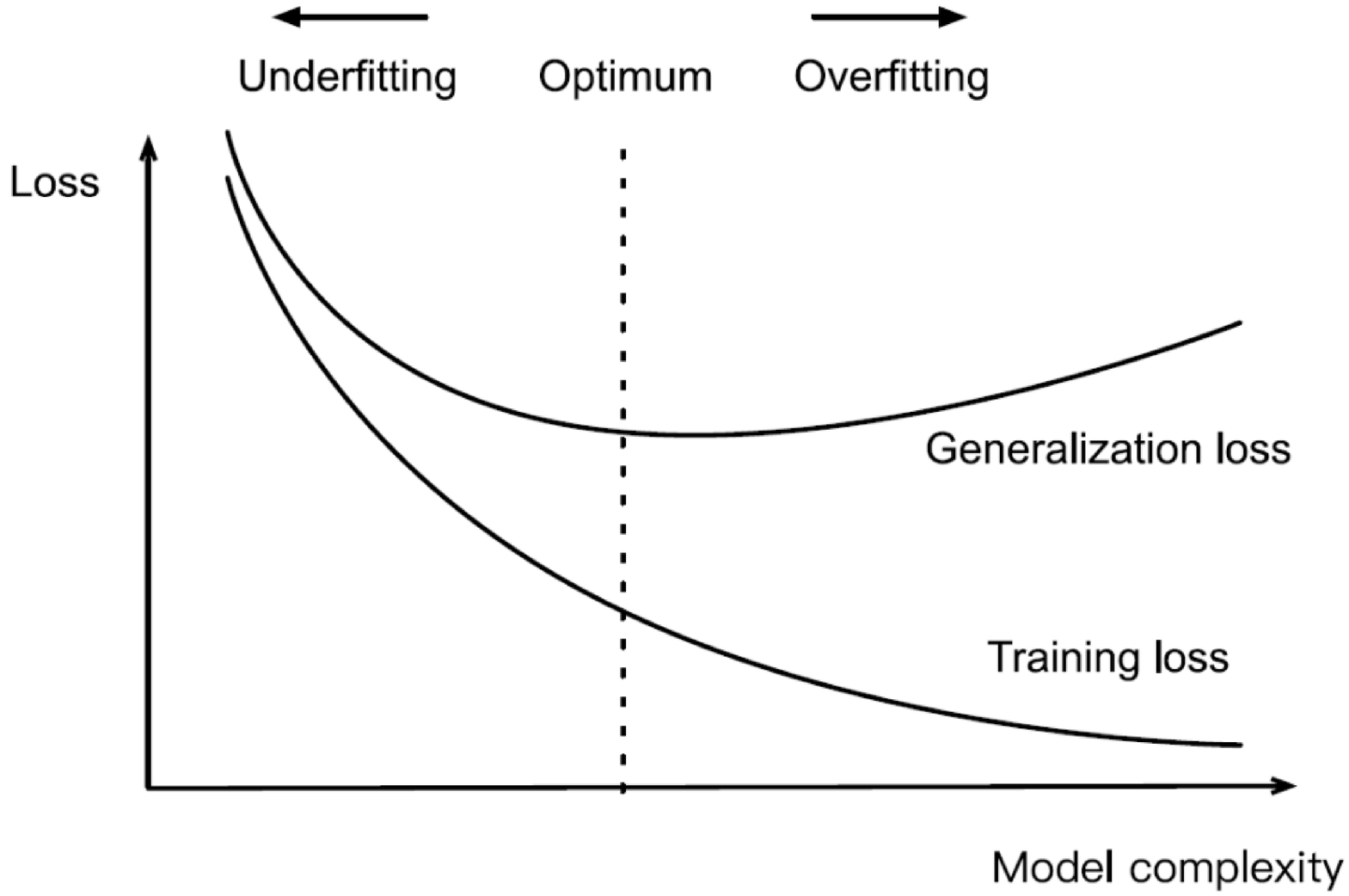



Figure 11: Influence of model complexity on underfitting and overfitting.

# 6.3 Model selection, underfitting, and overfitting

- the other big consideration to bear in mind is the *dataset size*
- fixing our model, the fewer samples we have in the training dataset, the more likely (and more severely) we are to encounter overfitting
- as we increase the amount of training data, the generalization error typically decreases
- moreover, in general, more data never hurts
- for a fixed task and data distribution, there is typically a relationship between model complexity and dataset size

# 6.3 Model selection, underfitting, and overfitting

- given more data, we might profitably attempt to fit a more complex model
- without sufficient data, simpler models may be more difficult to beat
- for many tasks, deep learning only outperforms linear models when many thousands of training examples are available
- in part, the current success of deep learning owes to the current abundance of massive datasets due to Internet companies, cheap storage, connected devices, and the broad digitization of the economy

# 6.4 Weight decay

- now that we have characterized the problem of overfitting, we can introduce some standard techniques for regularizing models
- recall that we can always mitigate overfitting by going out and collecting more training data
- that can be costly, time consuming, or entirely out of our control, making it impossible in the short run
- for now, we can assume that we already have as much high-quality data as our resources permit and focus on regularization techniques

# 6.4 Weight decay

- *weight decay* (generally called $\ell_2$ regularization), might be the most widely-used technique for regularizing parametric machine learning models
- the technique is motivated by the basic intuition that the *simplest* models are the ones for which some norm of their weight vector is as small as possible
- the most common method for ensuring a small weight vector is to add its norm as a penalty term to the problem of minimizing the loss
- thus, we replace our original objective, *minimizing the prediction loss on the training labels*, with the new objective, *minimizing the sum of the prediction loss and the penalty term*
- now, if our weight vector grows too large, our learning algorithm might focus on minimizing the weight norm vs. minimizing the training error, which is exactly what we want

# 6.4 Weight decay

- for example, consider the mean squared error loss given by:

$$\mathcal{L}(\boldsymbol{w}) = \frac{1}{n}\sum_{i=1}^{n}\frac{1}{2}(\hat{y}^{(i)} - y^{(i)})^2.$$

- to penalize the size of the weight vector, we must somehow add the weight norm to the loss function, but how should the model trade off the standard loss for this new additive penalty?
- in practice, we characterize this trade-off via the *regularization constant* $\lambda$, a nonnegative hyperparameter that we fit using validation data
- for the one-hidden-layer MLP discussed above, the new objective becomes:

$$\mathcal{L}(\boldsymbol{w}) + \frac{\lambda}{2}\left(||\boldsymbol{W}^{(1)}||_F^2 + ||\boldsymbol{W}^{(2)}||_F^2\right),$$

where the Frobenius norm is a generalization for matrices of the Euclidean $\ell_2$ norm, and is defined as: $||\boldsymbol{W}||_F = \sqrt{\text{Tr}(\boldsymbol{W}^\top \boldsymbol{W})}$

# 6.4 Weight decay

- for $\lambda = 0$, we recover our original loss function
- for $\lambda > 0$, we restrict the size of $||\boldsymbol{W}^{(1)}||_F^2 + ||\boldsymbol{W}^{(2)}||_F^2$, the norm of the weights of our network
- we divide by 2 by convention: when we take the derivative of a quadratic function, the 2 and $1/2$ cancel out, ensuring that the expression for the update looks nice and simple
- we might wonder why we work with the squared norm and not the standard norm (i.e., the Euclidean $\ell_2$ norm); we do this for computational convenience
- by squaring the $\ell_2$ norm, we remove the square root, leaving the sum of squares of each component of the weight vector
- this makes the derivative of the penalty easy to compute: the sum of derivatives equals the derivative of the sum

# 6.4 Weight decay

- now, when applying the mini-batch stochastic gradient descent algorithm, in addition to updating the weights based on the amount by which the prediction differs from the actual label, we also shrink the size of the weights $\boldsymbol{W}^{(1)}$ and $\boldsymbol{W}^{(2)}$ towards zero
- that is why the method is sometimes called “weight decay”: given the penalty term alone, our optimization algorithm *decays* the weight at each step of training
- smaller values of $\lambda$ correspond to less constrained weights, whereas larger values of $\lambda$ constrain the weights more considerably

# 6.5 Dropout

- we introduced the classical approach to regularizing statistical models by penalizing the $\ell_2$ norm of the weights
- in probabilistic terms, we justified this technique by arguing that we have assumed a prior belief that weights take values from a Gaussian distribution with mean zero
- more intuitively, we might argue that we encouraged the model to spread out its weights among many features, rather than depending too much on a small number of potentially false associations

# 6.5 Dropout

- faced with more features than examples, linear models tend to overfit
- but given more examples than features, we can generally count on linear models not to overfit
- unfortunately, the reliability with which linear models generalize comes at a cost
- naively applied, linear models do not take into account interactions among features
- for every feature, a linear model must assign either a positive or a negative weight, ignoring context

# 6.5 Dropout

- this fundamental tension between generalizability and flexibility was described as the *bias-variance trade-off*
- linear models have high bias: they can only represent a small class of functions
- however, these models have low variance: they give similar results across different random samples of the data
- deep neural networks inhabit the opposite end of the bias-variance spectrum
- unlike linear models, neural networks are not confined to looking at each feature individually
- they can learn interactions among groups of features

# 6.5 Dropout

- even when we have far more examples than features, deep neural networks are capable of overfitting
- in 2017, a group of researchers demonstrated the extreme flexibility of neural networks by training deep nets on randomly-labeled images
- despite the absence of any true pattern linking the inputs to the outputs, they found that the neural network optimized by stochastic gradient descent could label every image in the training set perfectly
- consider what this means: if the labels are assigned uniformly at random and there are 10 classes, then no classifier can do better than 10% accuracy on holdout data
- the generalization gap here is a whopping 90%
- if our models are so expressive that they can overfit this badly, then when should we expect them not to overfit?

# 6.5 Dropout

- let us think briefly about what we expect from a good predictive model
- we want it to peform well on unseen data
- classical generalization theory suggests that, to close the gap between train and test performance, we should aim for a *simple* model
- simplicity can come in the form of a small number of dimensions
- additionally, as we saw when discussing weight decay ($\ell_2$ regularization), the norm of the weights also represents a useful measure of simplicity
- another useful notion of simplicity is *smoothness*, i.e., that the function should not be sensitive to noise
- for instance, when we classify images, we would expect that adding some random noise to the pixels should be mostly harmless

# 6.5 Dropout

- thus, we could inject noise into each layer of the network, before calculating the subsequent layer during training
- when training a deep network with many layers, injecting noise enforces smoothness on the input-output mapping
- this idea, called *dropout*, involves injecting noise while computing each internal layer during forward propagation, and it has become a standard technique for training neural networks
- the method is called *dropout* because we literally *drop out* some neurons during training
- throughout training, on each iteration, standard dropout consists of zeroing out some fraction of the nodes in each layer, before calculating the subsequent layer

# 6.5 Dropout

- we may argue that neural network overfitting is characterized by a state in which each layer relies on a specific pattern of activations in the previous layer, this condition being called *co-adaptation*
- dropout breaks up co-adaptation, by not allowing neurons to be too reliant on each other
- in standard dropout regularization, with *dropout probability p*, each intermediate activation $h$ is replaced by a random variable $h'$ as follows:

$$h' = \begin{cases} 0, & \text{with probability } p \\ \frac{h}{1-p}, & \text{otherwise} \end{cases}.$$

- thus, each layer is normalized by the fraction of nodes that were retained (not dropped out), and so the expectation remains unchanged, i.e., $E[h'] = h$

# 6.5 Dropout

- recall the MLP with a hidden layer and 5 hidden units in Figure 4
- when we apply dropout to a hidden layer, zeroing out each hidden unit with probability $p$, the result can be viewed as a network containing only a subset of the original neurons
- in Figure 12, $h_2$ and $h_5$ are removed
- consequently, the calculation of the outputs no longer depends on $h_2$ or $h_5$, and their respective gradient also vanishes when performing backpropagation
- in this way, the calculation of the output layer cannot be overly dependent on any one element of $h_1, \ldots, h_5$

# 6.5 Dropout

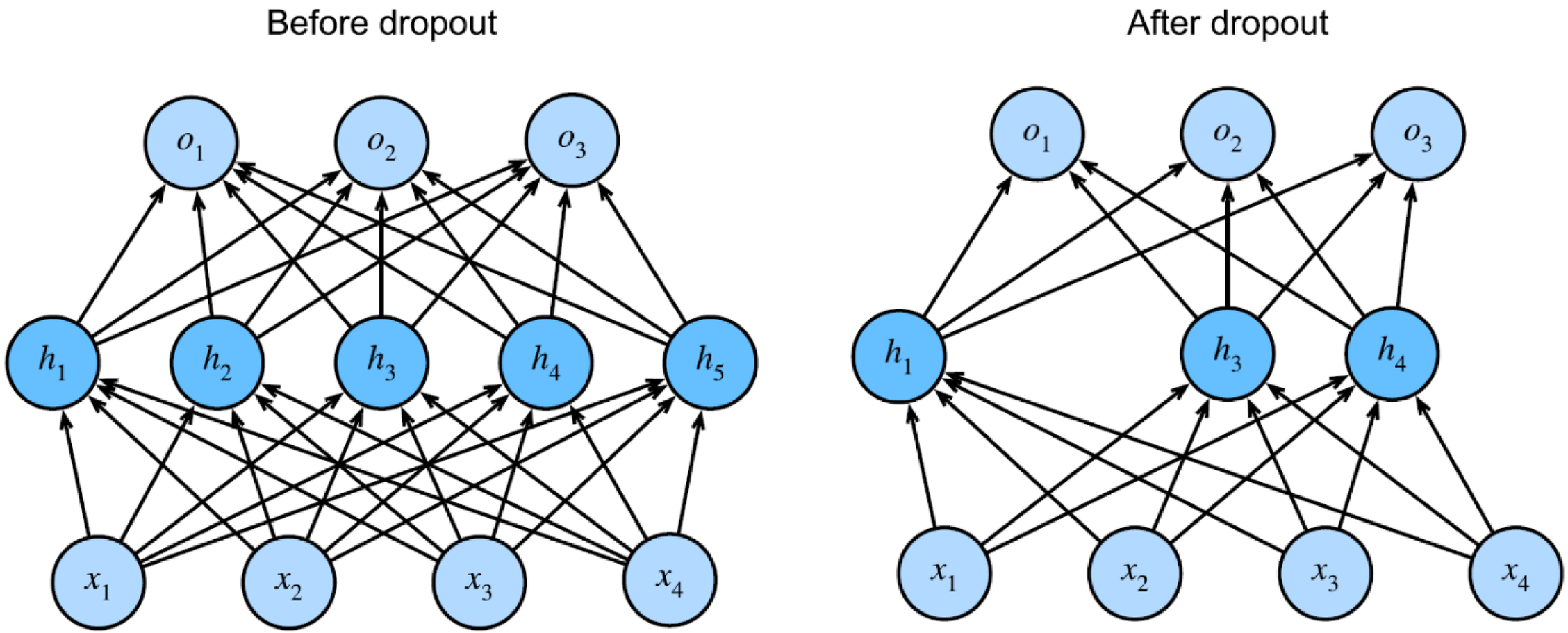



Figure 12: MLP before and after dropout.

# 6.5 Dropout

- dropout is disabled at test time: given a trained model and a new example, we do not drop out any nodes, and thus do not need to normalize
- to implement the dropout function for a single layer, we must draw as many samples from a Bernoulli (binary) random variable as our layer has dimensions, where the random variable takes value 1 (keep) with probability $1 - p$ and 0 (drop) with probability $p$
- one easy way to implement this is to first draw samples from the uniform distribution $\mathcal{U}(0, 1)$
- then we can keep those nodes for which the corresponding sample is greater than $p$, dropping the rest
- thus, we drop out the elements in the layer with probability $p$, rescaling the remainder as described above: dividing the survivors by $1 - p$

# 6.6 Forward propagation, backward propagation, and computational graphs

- when using modern deep learning frameworks, we only have to define the calculations involved in *forward propagation* through the model
- when it comes to calculating the gradients in order to perform gradient descent, we just invoke the backpropagation function provided by the deep learning framework
- the automatic calculation of gradients (*automatic differentiation*) profoundly simplifies the implementation of deep learning algorithms
- before automatic differentiation, even small changes to complicated models required recalculating complicated derivatives by hand
- surprisingly often, academic papers had to allocate numerous pages to deriving update rules
- while we must continue to rely on automatic differentiation so we can focus on the interesting parts, we should know how these gradients are calculated under the hood, if we want to go beyond a shallow understanding of deep learning

# 6.6 Forward propagation, backward propagation, and computational graphs

- in this section, we take a deep dive into the details of *backward propagation* (more commonly called *backpropagation*)
- to convey some insight for both the techniques and their implementations, we rely on some basic mathematics and computational graphs
- to start, we focus our exposition on a one-hidden-layer MLP with weight decay ($\ell_2$ regularization)
- *forward propagation* (or *forward pass*) refers to the calculation and storage of *intermediate variables* (including outputs) for a neural network in order, from the input layer to the output layer

# 6.6 Forward propagation, backward propagation, and computational graphs

- for the sake of simplicity, let us assume that the input example is $\boldsymbol{x} \in \mathbb{R}^d$ and that our hidden layer does not include a bias term
- here, the intermediate variable is:

$$\boldsymbol{z} = \boldsymbol{W}^{(1)}\boldsymbol{x},$$

  where $\boldsymbol{W}^{(1)} \in \mathbb{R}^{h \times d}$ is the weight parameter of the hidden layer
- after running the intermediate variable $\boldsymbol{z} \in \mathbb{R}^h$ through the activation function $\phi$, we obtain our hidden activation vector of length $h$:

$$\boldsymbol{h} = \phi(\boldsymbol{z}).$$

- the hidden variable $\boldsymbol{h} \in \mathbb{R}^h$ is also an intermediate variable
- assuming that the parameters of the output layer only possess a weight of $\boldsymbol{W}^{(2)} \in \mathbb{R}^{q \times h}$, we can obtain an output vector of length $q$:

$$\boldsymbol{o} = \boldsymbol{W}^{(2)}\boldsymbol{h}.$$

# 6.6 Forward propagation, backward propagation, and computational graphs

- assuming that the loss function is $l$, the squared error loss, and the example label is $\boldsymbol{y} \in \mathbb{R}^q$, we can then calculate the loss term for a single data example as:

$$L = l(\boldsymbol{o}, \boldsymbol{y}) = \frac{1}{2}(\boldsymbol{o} - \boldsymbol{y})^\top (\boldsymbol{o} - \boldsymbol{y}).$$

- according to the definition of $\ell_2$ regularization, given the hyperparameter $\lambda$, the regularization term is:

$$s = \frac{\lambda}{2}\left(||\boldsymbol{W}^{(1)}||_F^2 + ||\boldsymbol{W}^{(2)}||_F^2\right), \tag{1}$$

  where the Frobenius norm is defined as: $||\boldsymbol{W}||_F = \sqrt{\text{Tr}(\boldsymbol{W}^\top \boldsymbol{W})}$

- finally, the model's regularized loss on a given data example is:

$$J = L + s.$$

- we refer to $J$ as the *objective function* in the following discussion

# 6.6 Forward propagation, backward propagation, and computational graphs

- plotting *computational graphs* helps us visualize the dependencies of operators and variables within the calculation
- Figure 13 contains the graph associated with the simple network described above, where squares denote variables and circles denote operators
- the lower-left corner signifies the input and the upper-right corner is the output
- notice that the directions of the arrows (which illustrate data flow) are primarily rightward and upward

# 6.6 Forward propagation, backward propagation, and computational graphs

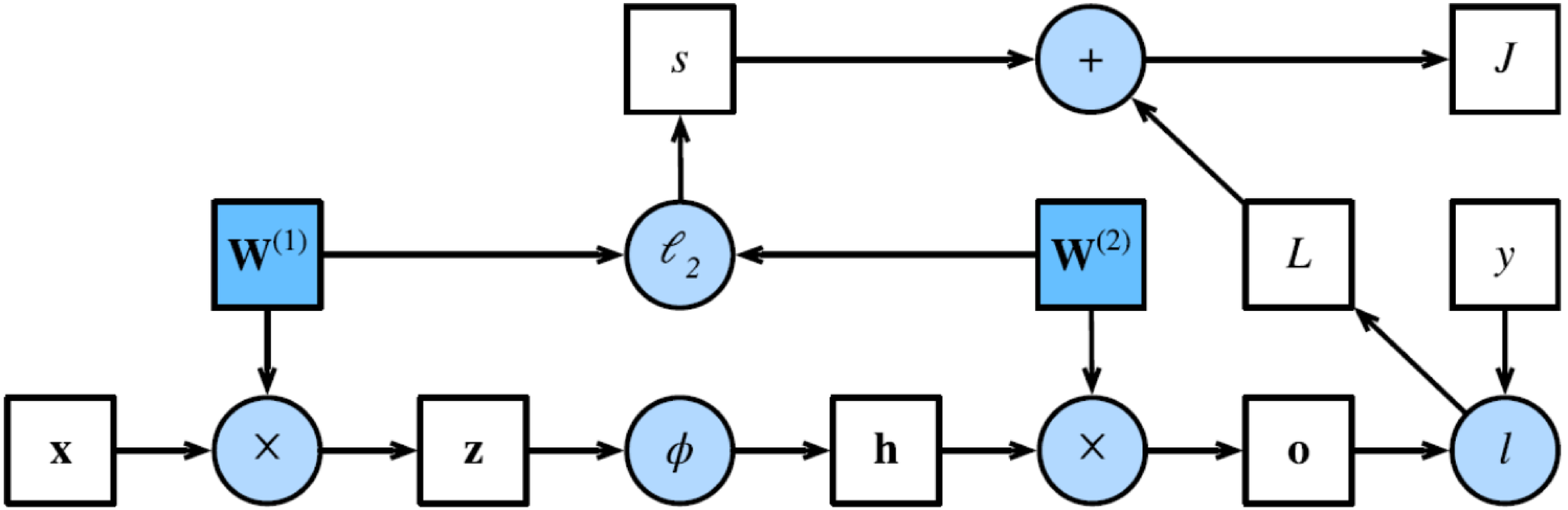



Figure 13: Computational graph of forward propagation.

# 6.6 Forward propagation, backward propagation, and computational graphs

- *backpropagation* refers to the method of calculating the gradient of neural network parameters
- in short, the method traverses the network in reverse order, from the output to the input layer, according to the *chain rule* from calculus
- the algorithm stores any intermediate variables (partial derivatives) required while calculating the gradient with respect to some parameters
- assume that we have functions $\mathsf{Y} = f(\mathsf{X})$ and $\mathsf{Z} = g(\mathsf{Y})$, in which the input and the output $\mathsf{X}, \mathsf{Y}, \mathsf{Z}$ are scalars, vectors, or matrices of arbitrary shapes

# 6.6 Forward propagation, backward propagation, and computational graphs

- by using the chain rule, we can compute the derivative of Z with respect to X via the chain rule as:

$$\frac{\partial \mathrm{Z}}{\partial \mathrm{X}} = \mathrm{prod}\left(\frac{\partial \mathrm{Z}}{\partial \mathrm{Y}}, \frac{\partial \mathrm{Y}}{\partial \mathrm{X}}\right).$$

- here, we use the prod operator to multiply its arguments after the necessary operations, such as transposition and swapping input positions, have been carried out
- for vectors, this is straightforward: it is simply matrix-matrix multiplication
- for matrices, we use the appropriate counterpart
- the operator prod hides all the notation overhead

# 6.6 Forward propagation, backward propagation, and computational graphs

- recall that the parameters of the simple network with one hidden layer, whose computational graph is in Figure 13, are $\boldsymbol{W}^{(1)}$ and $\boldsymbol{W}^{(2)}$
- the objective of backpropagation is to calculate the gradients $\frac{\partial J}{\partial \boldsymbol{W}^{(1)}}$ and $\frac{\partial J}{\partial \boldsymbol{W}^{(2)}}$
- to accomplish this, we apply the chain rule and calculate, in turn, the gradient of each intermediate variable and parameter
- the order of calculations are reversed relative to those performed in forward propagation, since we need to start with the outcome of the computational graph and work our way towards the parameters

# 6.6 Forward propagation, backward propagation, and computational graphs

- we start by calculating the gradient $\frac{\partial J}{\partial \boldsymbol{W}^{(2)}} \in \mathbb{R}^{q \times h}$ of the model parameters closest to the output layer
- first, we can write that:

$$\begin{aligned} \frac{\partial J}{\partial \boldsymbol{W}^{(2)}} &= \frac{\partial (L + s)}{\partial \boldsymbol{W}^{(2)}} \\ &= \frac{\partial L}{\partial \boldsymbol{W}^{(2)}} + \frac{\partial s}{\partial \boldsymbol{W}^{(2)}}. \end{aligned} \tag{2}$$

- using the chain rule for the first term in (2) yields:

$$\begin{aligned} \frac{\partial L}{\partial \boldsymbol{W}^{(2)}} &= \text{prod}\left(\frac{\partial L}{\partial \boldsymbol{o}}, \frac{\partial \boldsymbol{o}}{\partial \boldsymbol{W}^{(2)}}\right) \\ &= \text{prod}\left(\frac{\partial L}{\partial \boldsymbol{o}}, \frac{\partial (\boldsymbol{W}^{(2)} \boldsymbol{h})}{\partial \boldsymbol{W}^{(2)}}\right) \\ &= \frac{\partial L}{\partial \boldsymbol{o}} \boldsymbol{h}^{\top}. \end{aligned} \tag{3}$$

# 6.6 Forward propagation, backward propagation, and computational graphs

- now, we just have to compute the gradient of the loss function $L$ with respect to the variable of the output layer $\boldsymbol{o}$:

$$\begin{aligned}\frac{\partial L}{\partial \boldsymbol{o}} &= \frac{\partial}{\partial \boldsymbol{o}}\left(\frac{1}{2}(\boldsymbol{o}-\boldsymbol{y})^\top(\boldsymbol{o}-\boldsymbol{y})\right)\\ &= \boldsymbol{o}-\boldsymbol{y}.\end{aligned}$$

- thus, (3) becomes:

$$\frac{\partial L}{\partial \boldsymbol{W}^{(2)}} = (\boldsymbol{o}-\boldsymbol{y})\boldsymbol{h}^\top.$$

- we next calculate the gradient of the regularization term $s$ with respect to $\boldsymbol{W}^{(2)}$:

$$\frac{\partial s}{\partial \boldsymbol{W}^{(2)}} = \lambda \boldsymbol{W}^{(2)}.$$

- putting it all together, (2) becomes:

$$\frac{\partial J}{\partial \boldsymbol{W}^{(2)}} = (\boldsymbol{o}-\boldsymbol{y})\boldsymbol{h}^\top + \lambda \boldsymbol{W}^{(2)}.$$

# 6.6 Forward propagation, backward propagation, and computational graphs

- in order to obtain the gradient $\frac{\partial J}{\partial \boldsymbol{W}^{(1)}} \in \mathbb{R}^{h \times d}$ of the model parameters closest to the input layer, we first write that:

$$\begin{aligned} \frac{\partial J}{\partial \boldsymbol{W}^{(1)}} &= \frac{\partial (L + s)}{\partial \boldsymbol{W}^{(1)}} \\ &= \frac{\partial L}{\partial \boldsymbol{W}^{(1)}} + \frac{\partial s}{\partial \boldsymbol{W}^{(1)}}. \end{aligned} \tag{4}$$

- for the first term in (4), according to the chain rule, we get:

$$\begin{aligned} \frac{\partial L}{\partial \boldsymbol{W}^{(1)}} &= \text{prod}\left(\frac{\partial L}{\partial \boldsymbol{z}}, \frac{\partial \boldsymbol{z}}{\partial \boldsymbol{W}^{(1)}}\right) \\ &= \text{prod}\left(\frac{\partial L}{\partial \boldsymbol{z}}, \frac{\partial (\boldsymbol{W}^{(1)} \boldsymbol{x})}{\partial \boldsymbol{W}^{(1)}}\right) \\ &= \frac{\partial L}{\partial \boldsymbol{z}} \boldsymbol{x}^{\top}. \end{aligned}$$

# 6.6 Forward propagation, backward propagation, and computational graphs

- since the activation function $\phi$ applies element-wise, calculating the gradient $\frac{\partial L}{\partial \boldsymbol{z}} \in \mathbb{R}^h$ of the intermediate variable $\boldsymbol{z}$ requires that we use the element-wise matrix multiplication operator, which we denote by $\odot$:

$$\begin{aligned}\frac{\partial L}{\partial \boldsymbol{z}} &= \text{prod}\left(\frac{\partial L}{\partial \boldsymbol{h}}, \frac{\partial \boldsymbol{h}}{\partial \boldsymbol{z}}\right) \\ &= \text{prod}\left(\frac{\partial L}{\partial \boldsymbol{h}}, \frac{\partial \phi(\boldsymbol{z})}{\partial \boldsymbol{z}}\right) \\ &= \frac{\partial L}{\partial \boldsymbol{h}} \odot \phi'(\boldsymbol{z}). \end{aligned} \tag{5}$$

# 6.6 Forward propagation, backward propagation, and computational graphs

- the gradient with respect to the hidden layer's outputs $\frac{\partial L}{\partial \boldsymbol{h}} \in \mathbb{R}^h$ is given by the chain rule:

$$\begin{aligned}
\frac{\partial L}{\partial \boldsymbol{h}} &= \text{prod}\left(\frac{\partial L}{\partial \boldsymbol{o}}, \frac{\partial \boldsymbol{o}}{\partial \boldsymbol{h}}\right) \\
&= \text{prod}\left(\frac{\partial L}{\partial \boldsymbol{o}}, \frac{\partial (\boldsymbol{W}^{(2)}\boldsymbol{h})}{\partial \boldsymbol{h}}\right) \\
&= \boldsymbol{W}^{(2)\top}\frac{\partial L}{\partial \boldsymbol{o}} \\
&= \boldsymbol{W}^{(2)\top}(\boldsymbol{o} - \boldsymbol{y}).
\end{aligned}$$

# 6.6 Forward propagation, backward propagation, and computational graphs

- now, (5) becomes:

$$\frac{\partial L}{\partial \boldsymbol{z}} = \left(\boldsymbol{W}^{(2)\top}(\boldsymbol{o} - \boldsymbol{y})\right) \odot \phi'(\boldsymbol{z}).$$

- we similarly as before calculate the gradient of the regularization term $s$ with respect to $\boldsymbol{W}^{(1)}$:

$$\frac{\partial s}{\partial \boldsymbol{W}^{(1)}} = \lambda \boldsymbol{W}^{(1)}.$$

- putting it all together, (4) is finally given as:

$$\frac{\partial J}{\partial \boldsymbol{W}^{(1)}} = \left(\left(\boldsymbol{W}^{(2)\top}(\boldsymbol{o} - \boldsymbol{y})\right) \odot \phi'(\boldsymbol{z})\right) \boldsymbol{x}^{\top} + \lambda \boldsymbol{W}^{(1)}.$$

# 6.6 Forward propagation, backward propagation, and computational graphs

- thus, we take the following steps to compute $\frac{\partial J}{\partial \boldsymbol{W}^{(2)}}$:

1. Compute $\frac{\partial L}{\partial \boldsymbol{o}} = \boldsymbol{o} - \boldsymbol{y}$
2. Compute $\frac{\partial L}{\partial \boldsymbol{W}^{(2)}} = \frac{\partial L}{\partial \boldsymbol{o}} \boldsymbol{h}^\top$
3. Compute $\frac{\partial s}{\partial \boldsymbol{W}^{(2)}} = \lambda \boldsymbol{W}^{(2)}$
4. Compute $\frac{\partial J}{\partial \boldsymbol{W}^{(2)}} = \frac{\partial L}{\partial \boldsymbol{W}^{(2)}} + \frac{\partial s}{\partial \boldsymbol{W}^{(2)}}$

- and the following steps to compute $\frac{\partial J}{\partial \boldsymbol{W}^{(1)}}$:

1. Compute $\frac{\partial L}{\partial \boldsymbol{h}} = \boldsymbol{W}^{(2)\top} \frac{\partial L}{\partial \boldsymbol{o}}$ ($\frac{\partial L}{\partial \boldsymbol{o}}$ is already computed in step 1 above)
2. Compute $\frac{\partial L}{\partial \boldsymbol{z}} = \frac{\partial L}{\partial \boldsymbol{h}} \odot \phi'(\boldsymbol{z})$
3. Compute $\frac{\partial L}{\partial \boldsymbol{W}^{(1)}} = \frac{\partial L}{\partial \boldsymbol{z}} \boldsymbol{x}^\top$
4. Compute $\frac{\partial s}{\partial \boldsymbol{W}^{(1)}} = \lambda \boldsymbol{W}^{(1)}$
5. Compute $\frac{\partial J}{\partial \boldsymbol{W}^{(1)}} = \frac{\partial L}{\partial \boldsymbol{W}^{(1)}} + \frac{\partial s}{\partial \boldsymbol{W}^{(1)}}$

# 6.6 Forward propagation, backward propagation, and computational graphs

- when training neural networks, forward and backward propagation depend on each other
- in particular, for forward propagation, we traverse the computational graph in the direction of dependencies, and compute all the variables on its path
- these are then used for backpropagation, where the compute order on the graph is reversed
- take the aforementioned simple network as an example to illustrate
- on one hand, computing the regularization term (1) during forward propagation depends on the current values of model parameters $\boldsymbol{W}^{(1)}$ and $\boldsymbol{W}^{(2)}$
- they are given by the optimization algorithm according to backpropagation, in the latest iteration
- on the other hand, the gradient calculation for the parameter (3) during backpropagation depends on the current value of the hidden variable $\boldsymbol{h}$, which is given by forward propagation

# 6.6 Forward propagation, backward propagation, and computational graphs

- therefore, when training neural networks, after model parameters are *initialized*, we alternate *forward propagation* with *backpropagation*, *updating* model parameters using *gradients* given by backpropagation
- note that backpropagation reuses the stored intermediate values from forward propagation to avoid duplicate calculations
- one of the consequences is that we need to retain the intermediate values until backpropagation is complete
- this is also one of the reasons why training requires significantly more memory than plain prediction
- besides, the size of such intermediate values is roughly proportional to the number of network layers and the batch size
- thus, training deeper networks using larger batch sizes more easily leads to out of memory errors

# 6.7 Numerical stability and initialization

- thus far, every model that we discussed requires that we initialize its parameters according to some pre-specified distribution
- until now, we took the initialization scheme for granted, and didn't discuss the details of how these choices are made
- we might have even gotten the impression that these choices are not especially important
- to the contrary, the choice of initialization scheme plays a significant role in neural network learning, and it can be crucial for maintaining numerical stability

# 6.7 Numerical stability and initialization

- moreover, these choices can be tied up in interesting ways with the choice of the nonlinear activation function
- which function we choose and how we initialize parameters can determine how quickly our optimization algorithm converges
- poor choices here can cause us to encounter exploding or vanishing gradients while training
- in this section, we delve into these topics with greater detail and discuss some useful heuristics

# 6.7 Numerical stability and initialization

- consider a deep network with $L$ layers, input $\boldsymbol{x}$ and output $\boldsymbol{o}$
- with each layer $l$ defined by a transformation $f_l$ parameterized by weights $\boldsymbol{W}^{(l)}$, whose hidden variable is $\boldsymbol{h}^{(l)}$ (let $\boldsymbol{h}^{(0)} = \boldsymbol{x}$), our network can be expressed as:

$$\boldsymbol{h}^{(l)} = f_l(\boldsymbol{h}^{(l-1)}) \text{ and thus } \boldsymbol{o} = f_L \circ \ldots \circ f_1(\boldsymbol{x}),$$

  where $\circ$ denotes function composition: $f \circ g(\boldsymbol{x}) = f(g(\boldsymbol{x}))$
- if all the hidden variables and the input are vectors, we can write the gradient of $\boldsymbol{o}$ with respect to any set of parameters $\boldsymbol{W}^{(l)}$ as follows:

$$\frac{\partial \boldsymbol{o}}{\partial \boldsymbol{W}^{(l)}} = \underbrace{\frac{\partial \boldsymbol{h}^{(L)}}{\partial \boldsymbol{h}^{(L-1)}}}_{\boldsymbol{M}^{(L)} \stackrel{\text{def}}{=}} \cdot \ldots \cdot \underbrace{\frac{\partial \boldsymbol{h}^{(l+1)}}{\partial \boldsymbol{h}^{(l)}}}_{\boldsymbol{M}^{(l+1)} \stackrel{\text{def}}{=}} \cdot \underbrace{\frac{\partial \boldsymbol{h}^{(l)}}{\partial \boldsymbol{W}^{(l)}}}_{\boldsymbol{v}^{(l)} \stackrel{\text{def}}{=}}.$$

# 6.7 Numerical stability and initialization

- in other words, this gradient is the product of $L - l$ matrices $\boldsymbol{M}^{(L)} \cdot \ldots \cdot \boldsymbol{M}^{(l+1)}$ and the gradient vector $\boldsymbol{v}^{(l)}$
- thus, we are susceptible to the same problems of numerical underflow that often appear when multiplying together too many probabilities
- when dealing with probabilities, a common trick is to switch into log-space, i.e., shifting pressure from the mantissa to the exponent of the numerical representation
- unfortunately, our problem above is more serious: initially, the matrices $\boldsymbol{M}^{(l)}$ may have a wide variety of eigenvalues
- they might be small or large, and their product might be *very large* or *very small*

# 6.7 Numerical stability and initialization

- the risks posed by unstable gradients go beyond numerical representation
- gradients of unpredictable magnitude also threaten the stability of our optimization algorithms
- we may be facing parameter updates that are either:
  - excessively large, destroying our model (the *exploding gradient* problem);
  - excessively small (the *vanishing gradient* problem), rendering learning impossible as parameters hardly move on each update.
- the vanishing gradient problem is often caused by the choice of the activation function that is appended following each layer's linear operations
- historically, the sigmoid function $1/(1+\exp(-x))$ was popular, because it resembles a thresholding function
- since early artificial neural networks were inspired by biological neural networks, the idea of neurons that fire either *fully* or *not at all* (like biological neurons) seemed appealing
- let us take a closer look at the sigmoid to see why it can cause vanishing gradients

# 6.7 Numerical stability and initialization

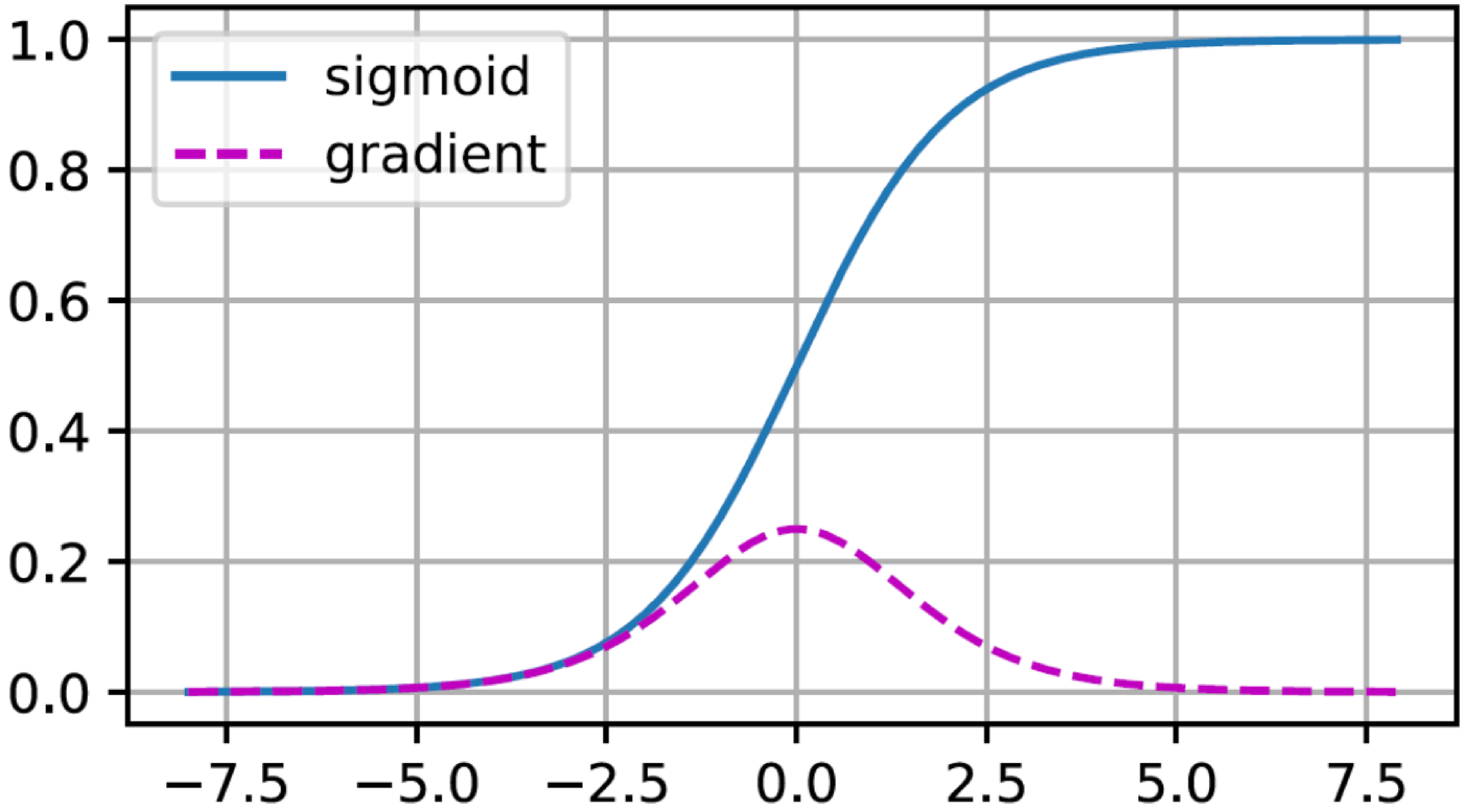



Figure 14: The sigmoid function and its derivative.

# 6.7 Numerical stability and initialization

- as we can see, the sigmoid's derivative vanishes both when its inputs are large and when they are small
- moreover, when backpropagating through many layers (unless we are in the zone where the input to the sigmoid is close to zero), the gradients of the overall product may vanish
- when our network has many layers, unless we are careful, the gradient will likely be cut off at some layer
- indeed, this problem used to plague deep network training
- consequently, ReLUs, which are more stable (but less neurally plausible), have emerged as the default choice today

# 6.7 Numerical stability and initialization

- the opposite problem, when gradients explode, can be similarly troublesome
- to illustrate this a bit better, we can draw 100 Gaussian random matrices and multiply them with some initial matrix
- if we choose the variance $\sigma^2 = 1$, we will see that the matrix product explodes
- when this happens due to the initialization of a deep network, we have no chance of getting a gradient descent optimizer to converge

# 6.7 Numerical stability and initialization

- another problem in neural network design is the symmetry inherent in their parametrization
- assume that we have a simple MLP with one hidden layer and two units
- in this case, we could permute the weights $\boldsymbol{W}^{(1)}$ of the first layer and likewise permute the weights of the output layer to obtain the same function
- there is nothing special differentiating the first hidden unit vs. the second hidden unit
- in other words, we have permutation symmetry among the hidden units of each layer

# 6.7 Numerical stability and initialization

- this is more than just a theoretical problem
- consider a one-hidden-layer MLP with two hidden units
- for illustration, suppose that the output layer transforms the two hidden units into only one output unit
- imagine what would happen if we initialized all of the parameters of the hidden layer as $\boldsymbol{W}^{(1)} = c$, for some constant $c$
- in this case, during forward propagation, either hidden unit takes the same inputs and parameters, producing the same activation, which is fed to the output unit

# 6.7 Numerical stability and initialization

- during backpropagation, differentiating the output unit with respect to parameters $\boldsymbol{W}^{(1)}$ gives a gradient whose elements all take the same value
- thus, after gradient-based iteration (e.g., mini-batch stochastic gradient descent), all the elements of $\boldsymbol{W}^{(1)}$ still take the same value
- such iterations would never break the symmetry on their own, and we might never be able to realize the network's expressive power
- the hidden layer would behave as if it had only a single unit
- note that, while mini-batch stochastic gradient descent would not break this symmetry, dropout regularization would

# 6.7 Numerical stability and initialization

- one way of addressing – or at least mitigating – the issues raised above is through careful initialization
- additional care during optimization and suitable regularization can further enhance stability
- let us look at the scale distribution of an output (e.g., a hidden variable) $o_i$ for some fully-connected layer *without nonlinearities*
- with $n_{\text{in}}$ inputs $x_j$ and their associated weights $w_{ij}$ for this layer, an output is given by:

$$o_i = \sum_{j=1}^{n_{\text{in}}} w_{ij} x_j.$$

# 6.7 Numerical stability and initialization

- the weights $w_{ij}$ are all drawn independently from the same distribution
- furthermore, let us assume that this distribution has zero mean and variance $\sigma^2$
- note that this does not mean that the distribution has to be Gaussian, just that the mean and variance need to exist
- for now, let us assume that the inputs to the layer $x_j$ also have zero mean and variance $\gamma^2$, and that they are independent of $w_{ij}$ and independent of each other
- in this case, we can compute the mean and variance of $o_i$ as follows:

# 6.7 Numerical stability and initialization

$$
\begin{aligned}
E[o_i] &= \sum_{j=1}^{n_{\text{in}}} E[w_{ij} x_j] \\
&= \sum_{j=1}^{n_{\text{in}}} E[w_{ij}] E[x_j] \\
&= 0, \\
\text{Var}[o_i] &= E[o_i^2] - (E[o_i])^2 \\
&= \sum_{j=1}^{n_{\text{in}}} E[w_{ij}^2 x_j^2] - 0 \\
&= \sum_{j=1}^{n_{\text{in}}} E[w_{ij}^2] E[x_j^2] \\
&= n_{\text{in}} \sigma^2 \gamma^2.
\end{aligned}
$$

# 6.7 Numerical stability and initialization

- one way to keep the variance fixed is to set $n_{\text{in}}\sigma^2 = 1$; now consider backpropagation
- there, we face a similar problem, although with gradients being propagated from the layers closer to the output
- using the same reasoning as for forward propagation, we see that the gradients' variance can explode, unless $n_{\text{out}}\sigma^2 = 1$, where $n_{\text{out}}$ is the number of outputs of this layer
- this leaves us in a dilemma: we cannot possibly satisfy both conditions simultaneously
- instead, we simply try to satisfy:

$$\frac{1}{2}(n_{\text{in}} + n_{\text{out}})\sigma^2 = 1 \text{ or, equivalently, } \sigma = \sqrt{\frac{2}{n_{\text{in}} + n_{\text{out}}}}.$$

# 6.7 Numerical stability and initialization

- this is the reasoning underlying the now-standard and practically beneficial *Xavier initialization*, named after the first author of its creators (Xavier Glorot)
- typically, the Xavier initialization samples weights from a Gaussian distribution with zero mean and variance $\sigma^2 = \frac{2}{n_{\text{in}}+n_{\text{out}}}$
- we can also adapt Xavier's intuition to choose the variance when sampling weights from a uniform distribution
- note that the uniform distribution $\mathcal{U}(-a, a)$ has variance $\frac{a^2}{3}$
- plugging $\frac{a^2}{3}$ into our condition on $\sigma^2$ yields the suggestion to initialize according to:

$$\mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}\right).$$

- though the assumption for nonexistence of nonlinearities in the above mathematical reasoning can be easily violated in neural networks, the Xavier initialization method turns out to work well in practice

# 6.8 Deep learning computation

- alongside giant datasets and powerful hardware, great software tools have played an indispensable role in the rapid progress of deep learning
- starting with the groundbreaking *Theano* library released in 2007, flexible open-source tools have enabled researchers to rapidly prototype models, avoiding repetitive work when recycling standard components, while still maintaining the ability to make low-level modifications
- over time, deep learning's libraries have evolved to offer increasingly coarse abstractions
- just as semiconductor designers went from specifying transistors to logical circuits to writing code, neural networks researchers have moved from thinking about the behavior of individual artificial neurons to conceiving of networks in terms of whole layers, and now often design architectures with far coarser *blocks* in mind

# 6.8 Deep learning computation

- when we introduced neural networks, we focused on linear models with a single output
- here, the entire model consists of just a single neuron
- note that a single neuron (i) takes some set of inputs; (ii) generates a corresponding scalar output; and (iii) has a set of associated parameters that can be updated to optimize some loss function of interest
- then, once we started thinking about networks with multiple outputs, we leveraged vectorized arithmetic to characterize an entire layer of neurons
- just like individual neurons, layers (i) take a set of inputs, (ii) generate corresponding outputs, and (iii) are described by a set of tunable parameters
- when we worked through softmax regression, a single layer was itself the model
- however, even when we subsequently introduced MLPs, we could still think of the model as retaining this same basic structure

# 6.8 Deep learning computation

- interestingly, for MLPs, both the entire model and its constituent layers share this structure
- the entire model takes in raw inputs (the features), generates outputs (the predictions), and possesses parameters (the combined parameters from all constituent layers)
- likewise, each individual layer ingests inputs (supplied by the previous layer), generates outputs (the inputs to the subsequent layer), and possesses a set of tunable parameters that are updated according to the signal that flows backwards from the subsequent layer

# 6.8 Deep learning computation

- while we might think that neurons, layers, and models give us enough abstractions, it turns out that we often find it convenient to speak about components that are larger than an individual layer, but smaller than the entire model
- for example, the ResNet-152 architecture, which is wildly popular in computer vision, possesses hundreds of layers
- these layers consist of repeating patterns of groups of layers; implementing such a network one layer at a time can grow tedious
- this concern is not just hypothetical – such design patterns are common in practice
- the ResNet architecture, mentioned above, won the 2015 ImageNet and COCO computer vision competitions for both recognition and detection and remains a go-to architecture for many vision tasks
- similar architectures, in which layers are arranged in various repeating patterns, are now ubiquitous in other domains, including natural language processing and speech

# 6.8 Deep learning computation

- to implement these complex networks, we introduce the concept of a neural network *block*
- a block could describe a single layer, a component consisting of multiple layers, or the entire model itself
- one benefit of working with the block abstraction is that they can be combined into larger artifacts, often recursively
- this is illustrated in Figure 15
- by defining code to generate blocks of arbitrary complexity on demand, we can write surprisingly compact code and still implement complex neural networks

# 6.8 Deep learning computation

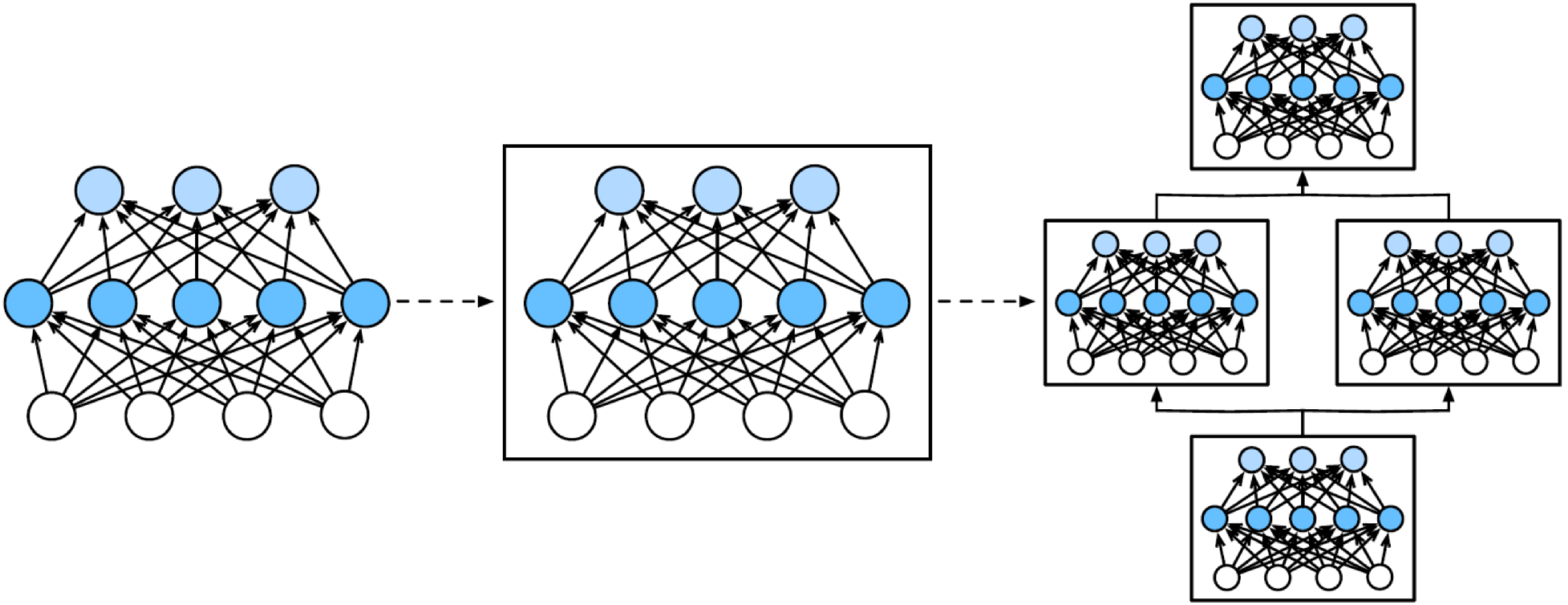


Figure 15: Multiple layers are combined into blocks, forming repeating patterns of larger models.

# 6.8 Deep learning computation

- from a programming point of view, a block is represented by a *class*
- any subclass of it must define a forward propagation function that transforms its input into output and must store any necessary parameters
- note that some blocks do not require any parameters at all
- finally, a block must possess a backpropagation function, for purposes of calculating gradients
- fortunately, due to the auto differentiation feature present in deep learning frameworks, when defining our own block, we only need to worry about parameters and the forward propagation function

# Exercises

## Exercise 6

Define an MLP for regression, having 3 inputs, one hidden layer of 2 units, with the ReLU activation function, and 1 output. For the input $\boldsymbol{X} = \begin{bmatrix} 1 & 0 & 1 \\ 1 & -1 & 0 \end{bmatrix}$, if the weight matrices and bias vectors of the layers are generated randomly having values from the set $\{-1, 0, 1\}$ (you can generate them yourself, not all zeros), compute the output $\boldsymbol{O}$ of the network. Specify the total number of real parameters (weights and biases) for the network.

**Solution:**

- we have the following formulas:

$$\begin{aligned} \boldsymbol{H} &= \sigma(\boldsymbol{X}\boldsymbol{W}^{(1)} + \boldsymbol{b}^{(1)}), \\ \boldsymbol{O} &= \boldsymbol{H}\boldsymbol{W}^{(2)} + \boldsymbol{b}^{(2)}, \end{aligned}$$

where $\boldsymbol{X} \in \mathbb{R}^{n \times d}$, $\boldsymbol{W}^{(1)} \in \mathbb{R}^{d \times h}$, $\boldsymbol{b}^{(1)} \in \mathbb{R}^{1 \times h}$, $\boldsymbol{H} \in \mathbb{R}^{n \times h}$, $\boldsymbol{W}^{(2)} \in \mathbb{R}^{h \times q}$, $\boldsymbol{b}^{(2)} \in \mathbb{R}^{1 \times q}$, $\boldsymbol{O} \in \mathbb{R}^{n \times q}$, and

$$\sigma(x) = \text{ReLU}(x) = \max(x, 0).$$

# Exercises

- in our case, $n = 2$, $d = 3$, $h = 2$, $q = 1$
- we take

$$\boldsymbol{W}^{(1)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & -1 \end{bmatrix} \in \mathbb{R}^{3\times 2},$$

$$\boldsymbol{b}^{(1)} = \begin{bmatrix} 1 & 0 \end{bmatrix} \in \mathbb{R}^{1\times 2},$$

$$\boldsymbol{W}^{(2)} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \in \mathbb{R}^{2\times 1},$$

$$\boldsymbol{b}^{(2)} = \begin{bmatrix} 1 \end{bmatrix} \in \mathbb{R}^{1\times 1}.$$

# Exercises

- now, we apply the formulas:

$$
\begin{aligned}
\boldsymbol{H} &= \sigma\left(\begin{bmatrix} 1 & 0 & 1 \\ 1 & -1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & -1 \end{bmatrix} + \begin{bmatrix} 1 & 0 \end{bmatrix}\right) \\
&= \sigma\left(\begin{bmatrix} 2 & -1 \\ 1 & -1 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix}\right) \\
&= \sigma\left(\begin{bmatrix} 3 & -1 \\ 2 & -1 \end{bmatrix}\right) \\
&= \begin{bmatrix} 3 & 0 \\ 2 & 0 \end{bmatrix} \in \mathbb{R}^{2\times 2},
\end{aligned}
$$

# Exercises

$$
\begin{aligned}
\boldsymbol{o} &= \begin{bmatrix} 3 & 0 \\ 2 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} + [1] \\
&= \begin{bmatrix} 3 \\ 2 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} \\
&= \begin{bmatrix} 4 \\ 3 \end{bmatrix} \in \mathbb{R}^{2\times 1},
\end{aligned}
$$

$$
\begin{aligned}
\text{Total parameters} &= 3 \times 2 + 1 \times 2 + 2 \times 1 + 1 \times 1 \\
&= 6 + 2 + 2 + 1 \\
&= 11.
\end{aligned}
$$

Vă mulţumesc!